

Orquestador de Seguridad: Fuzzing Automatizado de Aplicaciones Web

UNIVERSIDAD TECNOLÓGICA NACIONAL – FACULTAD REGIONAL MENDOZA

Tecnicatura Universitaria en Programación

Subtítulo: Sistema de orquestación para análisis automatizado de vulnerabilidades web mediante fuzzing y escaneo activo

Autores: Tomas Mastropietro, Cristian Krahulik, Juan Segura

Directores: Alberto Cortez y Ariel Enferrel.

Línea de Investigación: Seguridad informática · Ciberseguridad ofensiva · Automatización de pruebas · Ingeniería de software

Fecha: 2026

Resumen

El presente trabajo describe el diseño, implementación y validación de un sistema automatizado de análisis de seguridad para aplicaciones web basado en técnicas de fuzzing y escaneo activo. El sistema desarrollado, denominado Orquestador de Seguridad, fue implementado íntegramente en Python y permite coordinar la ejecución de múltiples herramientas especializadas de análisis de vulnerabilidades de manera automatizada y secuencial.

El sistema integra las herramientas OWASP ZAP, ffuf y SQLMap, cada una ejecutándose en entornos contenerizados mediante Docker. El orquestador central se encarga de coordinar la ejecución de cada herramienta, recolectar sus resultados, procesarlos mediante parsers especializados y consolidarlos en un único informe estructurado en formato JSON. La arquitectura implementada se basa en un modelo modular donde cada componente posee una responsabilidad específica: ejecución de herramientas externas, recolección de datos, procesamiento de resultados y consolidación final de hallazgos. Sobre esta base, el sistema se extendió hacia una arquitectura de servicio con API REST asíncrona, persistencia de historial, caché incremental, integración con n8n y un frontend en React.

Las pruebas del sistema fueron realizadas sobre DVWA (Damn Vulnerable Web Application), una aplicación web intencionalmente vulnerable utilizada comúnmente para investigación y entrenamiento en seguridad informática. Durante las pruebas se logró detectar múltiples vulnerabilidades relacionadas con el OWASP Top 10, incluyendo inyecciones SQL, configuraciones de seguridad incorrectas y rutas administrativas expuestas.

Los resultados obtenidos demuestran que la automatización del proceso de fuzzing y escaneo permite reducir el esfuerzo de análisis manual, facilitando la identificación temprana de vulnerabilidades en aplicaciones web. Estas comparaciones se detallan en el capítulo 14, junto con las limitaciones metodológicas correspondientes.

Palabras clave: Fuzzing, Seguridad Web, OWASP ZAP, SQLMap, Automatización, Docker, Python.

Abstract

This work presents the design, implementation and validation of an automated security analysis system for web applications based on fuzzing techniques and active vulnerability scanning. The developed system, called Security Orchestrator, was implemented entirely in Python and coordinates the execution of multiple specialized security tools in an automated and sequential pipeline.

The system integrates OWASP ZAP, ffuf, and SQLMap, each executed in containerized environments using Docker. The Python orchestrator manages tool execution, collects raw outputs, processes the results through specialized parsers, and consolidates all findings into a single structured JSON report.

The system was tested using DVWA (Damn Vulnerable Web Application) as a controlled laboratory target. The tests successfully identified vulnerabilities associated with the OWASP Top 10, including SQL injection, insecure configurations, and exposed administrative routes.

The results show that automated fuzzing orchestration improves vulnerability discovery efficiency and reduces manual testing effort in web security assessments, with the corresponding measurement caveats detailed in Chapter 14.

Keywords: Fuzzing, Web Security, OWASP ZAP, Automation, Vulnerability Scanning, Docker, Python.

Índice

Resumen	1
Abstract	2
Índice de Figuras	5
Índice de Tablas	5
1. Introducción	6
2. Planteo del Problema	7
3. Justificación	9
4. Objetivos	10
4.1 Objetivo General	10
4.2 Objetivos Específicos	10
5. Preguntas de Investigación	10
6. Hipótesis	11
7. Estado del Arte	11
8. Marco Teórico y Conceptual	14
8.1 Fuzzing	14
8.2 Escaneo de Vulnerabilidades	14
8.3 Contenedores Docker	14
8.4 OWASP Top 10	14
8.5 Pipeline de Datos y Arquitectura Modular	14
8.6 Orquestación y Automatización de Seguridad (SOAR)	15
9. Alcances y Limitaciones	16
9.1 Alcances	16
9.2 Limitaciones	16
10. Metodología	17
10.1 Diseño de la Investigación	17
10.2 Fases del Proyecto	17
10.3 Variables	17
10.4 Instrumentos	18
10.5 Amenazas a la Validez	18
10.6 Criterios de Validación	19
11. Arquitectura Propuesta e Implementada	20
11.1 Descripción General	20
11.2 Componentes y Sus Roles	21
11.3 Red Docker y Aislamiento	21
11.4 Flujo de Datos Detallado	22
11.5 Diagrama de Secuencia — Ciclo de Vida de POST /scan	23
12. Implementación Técnica	27
12.1 Infraestructura Docker	27
12.2 Pipeline de Ejecución	27
12.3 Módulo Runner	27
12.4 Módulo de Escaneo ZAP	27
12.5 Módulo de Escaneo ffuf	27
12.6 Módulo de Escaneo SQLMap	27
12.7 Sistema de Parseo	28
12.8 Consolidación de Resultados	28
12.9 Autenticación Automática	28
12.10 API REST y Frontend	29
13. Resultados Obtenidos	29
13.1 Resumen de Hallazgos	30

13.2 Vulnerabilidades Detectadas por OWASP ZAP	30
13.3 Rutas Descubiertas por ffuf	32
13.4 Análisis Automatizado con SQLMap	32
13.5 Análisis por Categoría OWASP Top 10	33
14. Discusión	34
14.1 Contraste de cobertura: uso individual frente a combinado (H2)	34
14.2 Reducción de esfuerzo manual (H1)	36
14.3 Verificación de H4: impacto del caché incremental	36
14.4 Rendimiento y ajuste de timeouts	38
14.5 Seguimiento longitudinal y sugerencias de mitigación por IA	38
14.6 Comparación con el panorama de la industria	38
15. Conclusiones y Trabajos Futuros	39
15.1 Conclusiones	39
15.1.1 Verificación por objetivo específico	41
15.2 Trabajos Futuros	41
16. Consideraciones Éticas	42
17. Desarrollo Experimental y Validación de Hallazgos	43
17.1 Cronograma y Distribución Temporal del Proceso de Auditoría	43
17.2 Procedimientos de Descubrimiento y Validación Manual por Vector de Ataque	44
17.2.1 Fase de Reconocimiento y Mapeo de Superficie (ZAP Spider y ffuf)	44
17.2.2 Inyección de Comandos del Sistema Operativo (ZAP Active Scan)	44
17.2.3 Vulnerabilidad de Inyección SQL (sqlmap)	44
17.2.4 Cross-Site Scripting Reflejado (ZAP Active Scan)	45
17.2.5 Inclusión Local de Archivos (ZAP Active Scan)	45
17.3 Conclusiones sobre la Metodología de Validación	45
18. Referencias	47
19. Anexos Técnicos	49
Anexo A — Estructura del Proyecto y configuración de docker compose	49
A.1 Estructura del proyecto	49
A.2 Archivo docker-compose.yml	50
A.3 Archivo Dockerfile	52
Anexo B — Fragmento Representativo: Pipeline de Ejecución	53
Anexo C — Fragmento Representativo: Parser de ZAP	54
Anexo D — Ejemplo del JSON Unificado Final (corrida del 5 de agosto de 2026)	55
Anexo E — Fragmento Representativo: Autenticación Automática	56
Anexo F — Fragmento Representativo: API REST	57
Anexo G — Frontend: Panel de Control	58
Anexo H — Diagramas en Tamaño Ampliado	59

Índice de Figuras

Figura 1. Diagrama de Arquitectura de Servicio	20
Figura 2. Diagrama Orquestador-Seguridad	21
Figura 3. Diagrama de Pipeline de ejecución	23
Figura 4. Diagrama de secuencia — ciclo de vida de POST /scan	24
Figura 1 (ampliada). Diagrama de Arquitectura de Servicio	59
Figura 2 (ampliada). Diagrama Orquestador-Seguridad	60
Figura 3 (ampliada). Diagrama del Pipeline de Ejecución	61
Figura 4 (ampliada). Diagrama de secuencia UML	62

Índice de Tablas

Tabla 1. Comparación de plataformas de orquestación y gestión de vulnerabilidades	13
Tabla 2. Componentes del sistema y su función en el pipeline	21
Tabla 3. Resumen de hallazgos — ejecución sobre DVWA del 5 de agosto de 2026	30
Tabla 4. Vulnerabilidades detectadas por OWASP ZAP, por tipo	31
Tabla 5. Rutas descubiertas por fuzzing de directorios (ffuf)	32
Tabla 6. Endpoints confirmados como vulnerables por SQLMap — corrida del 5 de agosto de 2026	32
Tabla 7. Datos extraídos a través de Sql Injection – corrida del 5 de agosto del 2026	32
Tabla 8. Hallazgos clasificados por categoría OWASP Top 10	33
Tabla 9. Cobertura por herramienta en solitario frente al pipeline combinado	35
Tabla 10. Tiempo total del pipeline con y sin caché incremental	36
Tabla 11. Estabilidad de hallazgos entre la corrida sin caché y con caché	37
Tabla 12. Verificación de cumplimiento por objetivo específico	41
Tabla 13. Distribución temporal de la sesión de auditoría manual del capítulo 17	46

1. Introducción

Las aplicaciones web constituyen actualmente uno de los componentes más importantes de la infraestructura digital moderna. Empresas, instituciones educativas, organismos gubernamentales y plataformas comerciales dependen cada vez más de sistemas accesibles a través de internet para brindar servicios a usuarios finales.

Esta dependencia creciente tiene un correlato directo en el panorama de amenazas: según el Data Breach Investigations Report 2025 de Verizon, los ataques básicos a aplicaciones web (Basic Web Application Attacks) representaron el 12% de las brechas de seguridad analizadas en 2025, frente al 9% del año anterior, y el 88% de esos ataques involucró credenciales robadas (Verizon, 2025). El Cost of a Data Breach Report 2025 de IBM sitúa el costo global promedio de una brecha de seguridad en 4,44 millones de dólares (IBM, 2025). Estas cifras contextualizan, con datos verificables, la afirmación de que las aplicaciones web son un vector de riesgo relevante y creciente.

Entre las vulnerabilidades más comunes en este tipo de aplicaciones se encuentran las inyecciones SQL, el Cross Site Scripting (XSS), las configuraciones de seguridad incorrectas, la autenticación débil y la exposición de endpoints internos. Estas categorías están ampliamente documentadas por la comunidad de seguridad informática, particularmente a través del proyecto OWASP Top 10, que identifica las categorías de riesgo más críticas para aplicaciones web.

Tradicionalmente, la detección de estas vulnerabilidades se realiza mediante auditorías de seguridad manuales o mediante herramientas especializadas de escaneo dinámico (DAST, Dynamic Application Security Testing), que analizan la aplicación en ejecución sin acceso a su código fuente (Qadir et al., 2025). El uso individual de estas herramientas requiere conocimientos técnicos avanzados y un proceso manual de correlación de resultados entre herramientas heterogéneas. Frente a esta problemática surge la necesidad de soluciones que integren múltiples herramientas dentro de un flujo coordinado, un enfoque que en la industria se enmarca dentro de lo que se conoce como SOAR (Security Orchestration, Automation and Response): la orquestación de herramientas de seguridad, la automatización de tareas repetitivas y la centralización de la respuesta ante hallazgos (Kinyua & Awuah, 2021).

Este trabajo integra tres herramientas que, en conjunto, cubren vectores de ataque complementarios: OWASP ZAP realiza escaneo dinámico general (pasivo y activo) sobre la aplicación en ejecución; ffuf se especializa en el descubrimiento de rutas, directorios y endpoints no enlazados mediante fuzzing de directorios (Alsaedi et al., 2021); y SQLMap se enfoca específicamente en la detección y explotación de inyecciones SQL, un tipo de vulnerabilidad que los escáneres generales suelen identificar con menor profundidad que una herramienta dedicada (Elia et al., 2010). La combinación de una herramienta de propósito general, una de descubrimiento de superficie de ataque y una especializada en un vector crítico busca maximizar la cobertura sin incurrir en la complejidad de plataformas comerciales de mayor escala.

Trabajos previos sobre orquestación de seguridad se discuten en detalle en el Capítulo 7 (Estado del Arte), donde se contrastan plataformas de código abierto orientadas a la gestión y orquestación de resultados de seguridad —Faraday, DefectDojo, TheHive y Shuffle— con la propuesta de este trabajo (ver Tabla 1). La brecha específica que este proyecto busca cubrir es la ausencia, entre esas alternativas, de una opción liviana que no requiera infraestructura compleja de despliegue, orientada puntualmente al escaneo web interactivo mediante fuzzing y escaneo activo, y desplegable en su totalidad con un único comando de Docker Compose.

En este contexto se desarrolla el presente proyecto, cuyo objetivo es diseñar e implementar un orquestador automatizado de herramientas de seguridad web capaz de ejecutar distintos tipos de análisis, normalizar los resultados heterogéneos producidos por cada herramienta y generar reportes unificados de vulnerabilidades detectadas. A partir de esta base, el sistema se extendió hacia una

arquitectura de servicio: una API REST que ejecuta los análisis de forma asíncrona, persiste el historial de escaneos y expone los resultados a un panel web, con soporte de autenticación automática para escanear secciones protegidas y de sugerencias de mitigación asistidas por inteligencia artificial. El sistema fue probado en un entorno de laboratorio controlado utilizando DVWA como aplicación objetivo, garantizando un marco ético y reproducible para la investigación.

Estructura del documento: El presente trabajo se encuentra articulado de la siguiente manera: las secciones 2 a 6 definen la problemática, justificación, metas, interrogantes e hipótesis orientadoras. Las bases teóricas y el estado del arte se examinan en las secciones 7 y 8, dando paso a la delimitación y metodología en los apartados 9 y 10. La arquitectura del sistema, el modelo de servicio y la implementación técnica del pipeline se detallan en los capítulos 11 y 12. La presentación de hallazgos y su debate técnico se abordan en las

secciones 13 y 14, condensando las conclusiones y líneas de investigación futuras en el capítulo 15. Por último, la sección 16 aborda las consideraciones éticas y normativas, el capítulo 17 profundiza el desarrollo experimental y los apartados 18 y 19 reúnen las fuentes bibliográficas y anexos técnicos.

2. Planteo del Problema

Pese a la amplia oferta de utilidades destinadas al diagnóstico de seguridad en plataformas web, la mayoría de las entidades tropieza con serias trabas operativas al momento de establecer un monitoreo sistemático y continuo. Como se argumentó previamente, los ataques directos contra vectores web representaron el 12% de los incidentes de seguridad registrados durante 2025 (Verizon, 2025), lo que evidencia la urgencia de abordar este problema mediante métricas y datos verificables.

Dentro del paradigma contemporáneo de desarrollo de software, caracterizado por metodologías ágiles e integración continua (CI/CD), la evaluación de riesgos suele quedar desplazada. Esto obedece a que los escáneres dinámicos (DAST) tradicionales fueron concebidos para ejecutarse de forma aislada, desarticulada y manual. Cada herramienta genera salidas heterogéneas y demanda parámetros independientes, originando complejas limitaciones metodológicas y arquitectónicas:

1. Silos de información e ineficiencia de integración: Los

componentes operan en entornos estancos y producen reportes disímiles (estructuras JSON incompatibles, salidas en texto plano o esquemas XML). Esta dispersión imposibilita la correlación automatizada e impide que los descubrimientos de superficie obtenidos por un fuzzer alimenten dinámicamente el análisis activo de otros escáneres.

2. Carga operativa por tareas repetitivas: Los auditores deben

iniciar manualmente cada utilidad de manera secuencial, monitorear su progreso de forma independiente y consolidar los hallazgos artesanalmente, insumiendo valiosas horas de trabajo especializado en labores administrativas de bajo impacto.

3. Extensión de los tiempos de análisis y fricción con CI/CD:

La ejecución sin coordinación de múltiples herramientas insume ventanas temporales incompatibles con la frecuencia de los despliegues modernos, escenario agravado por la carencia de mecanismos de caché que eviten evaluar reiteradamente endpoints sin cambios.

4. Deficiencias en la reproducibilidad del entorno: La

inconsistencia en las versiones, variables y configuraciones de ejecución dificulta contrastar los resultados de diferentes corridas, obstaculizando la verificación efectiva del proceso de remediación de hallazgos.

5. Brecha en la comunicación ejecutiva del riesgo: Los reportes

nativos se enfocan exclusivamente en detalles técnicos complejos, omitiendo síntesis de nivel directivo con clasificaciones globales de riesgo que orienten la toma de decisiones estratégicas.

6. Fatiga de alertas y saturación cognitiva: La acumulación de

logs desarticulados obliga a revisar extensos registros heterogéneos, incrementando la probabilidad de omitir vulnerabilidades críticas desestimadas entre un alto volumen de advertencias secundarias (Zhang et al., 2023).

Por lo tanto, se vuelve indispensable diseñar una solución de software que automatice la coordinación de herramientas de auditoría web, normalizando sus métricas bajo un esquema homogéneo. Dicho sistema debe constituir un orquestador liviano que impulse el enriquecimiento cruzado entre componentes, reduzca los tiempos de ejecución mediante persistencia incremental y elabore informes consolidados —técnicos y ejecutivos— evitando la complejidad estructural y los elevados costos de suscripción de las soluciones comerciales.

3. Justificación

La investigación se justifica desde múltiples dimensiones que se refuerzan mutuamente.

Desde el punto de vista académico, el proyecto permite integrar conocimientos adquiridos en áreas como programación, seguridad informática, arquitectura de software, contenedores Docker y, en su evolución, diseño de APIs y desarrollo frontend. Además, facilita la comprensión de cómo se realizan pruebas de seguridad en entornos reales y proporciona un caso de estudio concreto para la enseñanza de ciberseguridad ofensiva.

Desde una perspectiva técnica, la automatización del proceso de escaneo permite reducir errores humanos y mejorar la eficiencia en la detección de vulnerabilidades. La arquitectura modular adoptada demuestra que es posible integrar herramientas heterogéneas dentro de un pipeline cohesionado mediante patrones de diseño de software bien establecidos.

Desde una perspectiva económica, los procesos de evaluación de seguridad convencionales conllevan una erogación financiera sustancial para las entidades. Un análisis de penetración o auditoría web realizada manualmente requiere habitualmente de 20 a 40 horas de labor técnica cualificada, con honorarios profesionales que fluctúan entre 50 y 150 USD por hora en el mercado global (SANS Institute, 2024). En un escenario representativo de 30 horas a una tarifa media de 100 USD/h, una única auditoría externa demanda aproximadamente 3.000 USD, lo que para una organización con revisiones trimestrales implica un presupuesto anual cercano a los 12.000 USD. En el caso de pequeñas y medianas organizaciones, centros educativos o proyectos emergentes, este esquema resulta económicamente impracticable ante cada despliegue. Asimismo, las soluciones comerciales de gestión y orquestación reservan sus prestaciones avanzadas para esquemas de suscripción empresarial (DefectDojo, 2025). Frente a ello, el orquestador desarrollado —basado íntegramente en herramientas de código abierto y desplegable mediante Docker Compose— reduce el costo directo de licenciamiento a cero y disminuye el tiempo de ejecución a una ventana de entre 4 y 14 minutos, requiriendo únicamente una intervención humana acotada para el triaje final de hallazgos. Esto democratiza el diagnóstico preliminar continuo y promueve la adopción de prácticas de seguridad temprana (Shift-Left) sin incurrir en costos recurrentes de consultoría externa.

Finalmente, el proyecto contribuye a la transferencia de conocimiento, ya que demuestra cómo herramientas de seguridad ampliamente utilizadas en la industria pueden integrarse dentro de una arquitectura de software modular y automatizada —extendida hacia un servicio con API, persistencia e interfaz web—, generando un prototipo reproducible que puede ser adaptado por otros equipos de desarrollo o investigación.

4. Objetivos

4.1 Objetivo General

Diseñar e implementar un sistema automatizado que permita orquestar múltiples herramientas de análisis de seguridad web para detectar vulnerabilidades en aplicaciones web, consolidar los resultados en reportes unificados y exponer el sistema como un servicio reutilizable.

4.2 Objetivos Específicos

1. Integrar herramientas de análisis de seguridad web (OWASP ZAP, ffuf y SQLMap) en un flujo de ejecución automatizado, enriqueciendo el análisis de cada herramienta con los hallazgos de las demás.
2. Diseñar una arquitectura modular que permita extender el sistema fácilmente mediante la incorporación de nuevas herramientas o módulos de análisis.
3. Implementar parsers especializados que normalicen los resultados generados por cada herramienta en un formato homogéneo.
4. Desplegar el sistema en un entorno contenerizado utilizando Docker, garantizando la reproducibilidad del entorno de ejecución.
5. Evaluar el funcionamiento del sistema en un entorno de pruebas controlado (DVWA), verificando el cumplimiento de los criterios de validación definidos en la sección 10.6 (funcionalidad, reproducibilidad, cobertura mínima de tres categorías OWASP Top 10 y consolidación completa de hallazgos).
6. Implementar un mecanismo de autenticación automática que permita al orquestador acceder a secciones protegidas de la aplicación objetivo durante el escaneo.
7. Exponer el orquestador como un servicio mediante una API REST que ejecute los análisis de forma asíncrona, persista el historial de escaneos y permita su consulta posterior.
8. Generar automáticamente dos variantes de reporte —técnico y ejecutivo— con clasificación de riesgo global, orientadas a audiencias técnicas y no técnicas respectivamente.
9. Desarrollar una interfaz web que consuma la API del orquestador y permita lanzar escaneos, visualizar su progreso y consultar los hallazgos sin requerir acceso a la línea de comandos.

5. Preguntas de Investigación

Las siguientes preguntas guían el desarrollo y la evaluación del proyecto:

1. **(PI1)** ¿En qué medida un sistema de orquestación desarrollado en Python mejora, en términos cualitativos y de tiempo de re-ejecución, la coordinación de múltiples herramientas de seguridad web frente a su ejecución manual e independiente?
2. **(PI2)** ¿La integración de herramientas open source en un pipeline unificado —donde los hallazgos de una herramienta enriquecen el análisis de otra— permite ampliar la cobertura de detección respecto al uso individual de cada herramienta?
3. **(PI3)** ¿Es posible generar un formato unificado de resultados a partir de herramientas que producen salidas en formatos heterogéneos?
4. **(PI4)** ¿Qué ventajas ofrece una arquitectura modular y contenerizada para sistemas de análisis de seguridad automatizados en términos de extensibilidad y reproducibilidad?
5. **(PI5)** ¿Puede un sistema de este tipo extenderse, sin modificar su lógica de escaneo central, hacia un servicio persistente con interfaz web y capacidades de seguimiento histórico de vulnerabilidades?

6. Hipótesis

Se plantean las siguientes hipótesis de investigación:

H1: La automatización del proceso de análisis de seguridad mediante un sistema de orquestación reduce el esfuerzo manual necesario para detectar vulnerabilidades en aplicaciones web, al eliminar la necesidad de ejecutar cada herramienta de forma independiente y consolidar manualmente los resultados.

H2: La integración de múltiples herramientas de análisis (escaneo activo, fuzzing de directorios e inyección SQL), donde los hallazgos de una herramienta se utilizan para enriquecer el análisis de las demás, permite obtener una cobertura de detección mayor que el uso individual de cada herramienta.

H3: La normalización de resultados mediante parsers especializados facilita la interpretación de los hallazgos y mejora la utilidad de los reportes generados, al presentar información de fuentes heterogéneas en una estructura unificada y coherente.

H4: Un mecanismo de caché incremental respaldado en base de datos reduce el tiempo de las ejecuciones repetidas del pipeline sobre un mismo objetivo, al evitar repetir pruebas ya realizadas sobre elementos previamente analizados y sin hallazgos.

7. Estado del Arte

En los últimos años se han desarrollado numerosas herramientas destinadas al análisis de seguridad en aplicaciones web, clasificables en distintas categorías según su enfoque y funcionalidad.

Los escáneres de vulnerabilidades como OWASP ZAP o Burp Suite realizan análisis dinámico (DAST) mediante técnicas de escaneo activo y pasivo. Shahid et al. (2022), en un estudio comparativo de parámetros de seguridad en aplicaciones web publicado en Applied Sciences, analizan las tendencias actuales y las direcciones futuras de las herramientas de este tipo, señalando la fragmentación de enfoques entre escáneres de propósito general y herramientas especializadas como una de las limitaciones actuales del ecosistema (Shahid et al., 2022). En la misma línea, Qadir et al. (2025) evaluaron empíricamente nueve herramientas SAST y DAST de código abierto sobre setenta y cinco aplicaciones web reales, y encontraron que ninguna herramienta individual —estática o dinámica— cubre por sí sola la totalidad de las categorías del OWASP Top 10, lo que refuerza la necesidad de combinarlas (Qadir et al., 2025).

Althunayyan et al. (2022) evaluaron además cinco escáneres de caja negra frente a una aplicación de comercio electrónico moderna y encontraron diferencias sustanciales en la tasa de detección de vulnerabilidades de inyección entre herramientas, lo que respalda empíricamente la decisión de este proyecto de no depender de un único escáner (Althunayyan et al., 2022). En una línea directamente relevante para el diseño de este proyecto, Abdulghaffar, Elmrabit y Yousefi (2023) evaluaron un enfoque de pruebas de penetración automatizadas que combina múltiples escáneres de vulnerabilidades sobre un mismo objetivo y muestran que la combinación de herramientas heterogéneas amplía la cobertura de detección respecto del uso de un escáner aislado, un hallazgo que sustenta directamente la decisión arquitectónica de este trabajo de orquestar ZAP, ffuf y SQLMap en lugar de depender de una única herramienta (Abdulghaffar et al., 2023).

Las herramientas de fuzzing como ffuf o dirsearch permiten descubrir rutas ocultas o endpoints expuestos mediante el envío sistemático de requests generados a partir de wordlists, complementando el descubrimiento de superficie de ataque que realiza el spider de un escáner DAST. Alsaedi et al. (2021) relevan en su encuesta sobre fuzzing de caja negra el estado de las técnicas de construcción de ataques para aplicaciones web, y clasifican a las herramientas de descubrimiento de rutas como una etapa complementaria —no sustituta— del escaneo activo (Alsaedi et al., 2021). Zhang et al. (2023) proponen además un enfoque de escaneo sensible al estado de la aplicación que

reduce el tiempo desperdiciado en visitar páginas equivalentes durante el fuzzing, un problema de eficiencia análogo al que este proyecto aborda mediante el mecanismo de caché incremental descrito en la sección 11.1 (Zhang et al., 2023).

Por otra parte, herramientas especializadas como SQLMap se enfocan en la detección y explotación de un vector específico —inyección SQL— con mayor profundidad que la que ofrecen los escáneres generales para esa misma categoría de vulnerabilidad. Elia et al. (2010), en un estudio experimental comparativo de herramientas de detección de inyección SQL mediante inyección de ataques, encontraron que la efectividad de estas herramientas varía sustancialmente según el mecanismo de detección utilizado y advierten sobre los falsos negativos de los enfoques basados en coincidencia de texto —una observación directamente relevante para la limitación del parser de SQLMap de este proyecto, discutida en la sección 12.7 (Elia et al., 2010).

Más allá de estas herramientas puntuales, existen plataformas de código abierto orientadas específicamente a la orquestación y gestión de resultados de seguridad, generalmente agrupadas bajo el concepto de SOAR (Security Orchestration, Automation and Response).

Kinyua y Awuah (2021) caracterizan a las plataformas SOAR como la integración de herramientas de seguridad dispares, la automatización de tareas repetitivas de análisis y la estandarización de la respuesta ante hallazgos, y señalan como dirección de investigación futura la incorporación de componentes de aprendizaje automático para priorizar alertas (Kinyua & Awuah, 2021): Faraday centraliza los resultados de múltiples escáneres en un repositorio colaborativo; DefectDojo gestiona el ciclo de vida de vulnerabilidades con seguimiento histórico y métricas por proyecto; TheHive y Shuffle automatizan flujos de respuesta ante incidentes mediante playbooks configurables, de forma conceptualmente similar a como este proyecto usa n8n para disparar el análisis de sugerencias de mitigación. Estas plataformas resuelven el problema de integración y correlación de resultados que motiva este trabajo, pero requieren instalación y configuración considerablemente más complejas, y varias ofrecen sus funcionalidades avanzadas (gestión de equipos, integraciones empresariales) bajo modelos de licenciamiento comercial.

Tabla 1. Comparación de plataformas de orquestación y gestión de vulnerabilidades

Plataforma	Alcance funcional	Curva de instalación	Costo	Automatización del pipeline
Faraday	Centraliza resultados de múltiples escáneres en un repositorio colaborativo	Servidor dedicado y configuración de agentes/plugins	Community gratuito / Enterprise comercial	Centraliza resultados; no ejecuta ni coordina el escaneo en sí
DefectDojo	Gestión del ciclo de vida de vulnerabilidades, métricas por proyecto	Despliegue vía Docker Compose; requiere configurar importadores por herramienta	Community gratuito / Pro comercial	Importa resultados ya generados; no orquesta la ejecución de las herramientas
TheHive / Shuffle	Automatización de respuesta a incidentes mediante playbooks configurables	Infraestructura compuesta (TheHive + Cortex + integraciones)	Community gratuito / soporte comercial	Alto en respuesta a incidentes; no está orientado a fuzzing ni escaneo web
Orquestador de Seguridad (este proyecto)	Orquestación end-to-end de ZAP, ffuf y SQLMap con enriquecimiento cruzado entre herramientas	Un comando (docker compose up)	Gratuito, sin licenciamiento por niveles	Alto: coordina, enriquece y consolida automáticamente todo el pipeline de escaneo

Nota. Elaboración propia a partir de la documentación oficial de cada plataforma (Faraday Security, 2025; DefectDojo, 2025; TheHive Project, 2025; Shuffle, 2025) y de la descripción funcional del sistema desarrollado.

Frente a este panorama, el proyecto presentado en este trabajo propone una alternativa de menor escala: un orquestador propio en Python que integra tres herramientas puntuales, con persistencia de historial, API y un panel web simplificado, priorizando la simplicidad de despliegue y la reproducibilidad sobre la amplitud funcional de una plataforma SOAR completa.

8. Marco Teórico y Conceptual

8.1 Fuzzing

El fuzzing es una técnica de pruebas de seguridad que consiste en enviar entradas inesperadas o malformadas a un sistema con el objetivo de identificar comportamientos anómalos o vulnerabilidades (Alsaedi et al., 2021). Se distinguen tres enfoques según el nivel de conocimiento del sistema bajo prueba: black-box (sin conocimiento interno, basado únicamente en la interfaz expuesta), grey-box (con conocimiento parcial, por ejemplo instrumentación de cobertura) y white-box (con acceso completo al código fuente). La herramienta ffuf utilizada en este proyecto corresponde a un fuzzing black-box de tipo directory fuzzing: un subtipo específico de fuzzing web orientado al descubrimiento de rutas, distinto del fuzzing de parámetros o del fuzzing de protocolos de red y binarios (Alsaedi et al., 2021).

8.2 Escaneo de Vulnerabilidades

Se distingue entre DAST (Dynamic Application Security Testing), que analiza la aplicación en ejecución sin acceso a su código fuente —el enfoque utilizado por OWASP ZAP en este proyecto—, y SAST (Static Application Security Testing), que analiza el código fuente sin ejecutarlo. Qadir et al. (2025) comparan empíricamente ambos enfoques sobre setenta y cinco aplicaciones reales y concluyen que son complementarios antes que sustitutos: las herramientas SAST detectan mejor vulnerabilidades de severidad alta en etapas tempranas, mientras que las DAST —como las utilizadas en este proyecto— identifican fallas que solo se manifiestan en tiempo de ejecución (Qadir et al., 2025). Dentro del enfoque DAST se distinguen además el escaneo pasivo (análisis del tráfico sin modificar solicitudes) y el escaneo activo (envío de payloads para verificar vulnerabilidades).

8.3 Contenedores Docker

Docker permite ejecutar aplicaciones dentro de contenedores aislados que comparten el kernel del sistema operativo anfitrión, a diferencia de la virtualización completa, que emula hardware y ejecuta un sistema operativo independiente por cada máquina virtual. Para este proyecto, Docker se eligió por su menor sobrecarga de recursos y su tiempo de arranque casi instantáneo, adecuado para levantar y destruir entornos de laboratorio repetidamente. Esta elección tiene una implicancia de seguridad relevante: al compartir kernel, un contenedor mal configurado representa una superficie de escape potencial hacia el host. Javed y Toor (2021), en una evaluación de herramientas de detección de vulnerabilidades en contenedores (Clair, Anchore y Microscanner) sobre imágenes públicas de Docker Hub, encontraron que ninguna de las tres herramientas detecta de forma confiable vulnerabilidades a nivel de paquetes de aplicación —solo del sistema operativo base—, lo que refuerza por qué este proyecto no delega el análisis de seguridad de la aplicación al propio motor de contenedores, sino a herramientas dedicadas de DAST y fuzzing que operan sobre el servicio en ejecución (Javed & Toor, 2021). Por este motivo, los servicios de este proyecto se ejecutan sin privilegios extendidos.

8.4 OWASP Top 10

El proyecto OWASP publica periódicamente una lista de las diez categorías de riesgo más críticas para aplicaciones web (OWASP Foundation, 2021). Además de la inyección (A03:2021) y las fallas de configuración (A05:2021), el sistema desarrollado también detecta hallazgos de la categoría A01:2021 (Broken Access Control, mediante rutas administrativas descubiertas por ffuf) y, a través de sus componentes de autenticación, resulta relevante para A07:2021 (fallas de identificación y autenticación).

8.5 Pipeline de Datos y Arquitectura Modular

El pipeline implementado sigue el patrón arquitectónico conocido como Pipeline Pattern (o Pipes and Filters), en el que los datos fluyen a través de una secuencia de etapas de transformación, cada una con una responsabilidad única; este patrón fue formalizado por Buschmann, Meunier, Rohnert,

Sommerlad y Stal (1996) en su catálogo de patrones de arquitectura de software como una solución recurrente para sistemas que procesan datos en etapas independientes y reemplazables, propiedad que en este proyecto se aprovecha para intercalar la inyección de rutas de ffuf en la sesión de ZAP sin modificar el resto de las etapas del pipeline (Buschmann et al., 1996). La separación entre runners, scanners, parsers y workflow responde adicionalmente al principio de responsabilidad única (Single Responsibility Principle), uno de los cinco principios SOLID de diseño orientado a objetos sistematizados por Martin (2003), según el cual un módulo debe tener una única razón para cambiar; en este proyecto, esa separación es lo que permite sustituir o extender un scanner o un parser sin afectar al resto del pipeline (Martin, 2003).

8.6 Orquestación y Automatización de Seguridad (SOAR)

El concepto de SOAR (Security Orchestration, Automation and Response) engloba las tecnologías que permiten integrar herramientas de seguridad dispares, automatizar tareas repetitivas de análisis y estandarizar la respuesta ante hallazgos (Kinyua & Awuah, 2021). La extensión de este proyecto hacia una API con ejecución asíncrona, persistencia de historial y notificación mediante webhooks a un flujo de n8n lo posiciona conceptualmente dentro de esta categoría, aunque a una escala considerablemente más acotada que las plataformas SOAR comerciales o de código abierto de mayor madurez descritas en el Estado del Arte. Kinyua y Awuah (2021) identifican la incorporación de aprendizaje automático para la priorización de alertas como una de las principales direcciones futuras del campo; este proyecto no implementa dicha capacidad, pero sí incorpora un componente de sugerencias de mitigación asistidas por IA (sección 12.10), conceptualmente cercano a esa dirección de trabajo.

9. Alcances y Limitaciones

9.1 Alcances

El proyecto cubre el diseño, la implementación y la validación de un orquestador de seguridad funcional en un entorno de laboratorio controlado, con las siguientes capacidades implementadas y verificadas:

- Ejecución automatizada de tres herramientas de análisis de seguridad (OWASP ZAP, ffuf y SQLMap), con enriquecimiento cruzado entre ffuf y ZAP.
- Autenticación automática contra la aplicación objetivo, incluyendo renovación de sesión durante ejecuciones prolongadas.
- Descubrimiento de URLs mediante el spider de ZAP y fuzzing de directorios mediante ffuf con wordlists configurables.
- Detección de inyecciones SQL mediante SQLMap, con filtrado priorizado de URLs candidatas y ejecución en segundo plano.
- Parseo, normalización y consolidación de resultados en un archivo JSON unificado.
- Caché incremental de resultados y re-testeo automático de URLs previamente confirmadas como vulnerables.
- Generación automática de reportes técnico y ejecutivo, con clasificación de riesgo global.
- Exposición del orquestador mediante una API REST con ejecución asíncrona, seguimiento de progreso y persistencia de historial de escaneos.
- Interfaz web para lanzar escaneos, visualizar resultados y consultar sugerencias de mitigación asistidas por IA.
- Despliegue del entorno completo mediante Docker Compose.

9.2 Limitaciones

- El sistema fue probado únicamente sobre DVWA en un entorno de laboratorio; no se realizaron pruebas sobre aplicaciones en producción, por lo que la generalización de los resultados a otros contextos no está garantizada.
- La API no implementa autenticación ni restricción de origen (CORS abierto a cualquier dominio), por lo que su uso actual está limitado a entornos de laboratorio y no a un despliegue expuesto públicamente.
- Las sugerencias de mitigación generadas por el componente de IA no fueron validadas sistemáticamente contra un criterio experto; deben tratarse como apoyo complementario, no como fuente única de remediación.
- La clasificación de riesgo global del reporte ejecutivo utiliza una heurística simple (presencia de hallazgos High/SQL Injection), no un criterio estándar de la industria como CVSS.
- Debe señalarse la ausencia de una comparación empírica de alcance equivalente entre la ejecución del pipeline automatizado y un proceso de descubrimiento manual independiente; los registros del capítulo 17 corresponden a una sesión de validación y triaje sobre hallazgos ya detectados en nivel *Low*, lo que imposibilita la verificación controlada que exige H1 (véanse la sección 6 y el análisis metodológico en 14.2).
- No se cuantificó formalmente, mediante ejecuciones controladas de cada herramienta en aislamiento, la cobertura individual frente a la cobertura combinada; el capítulo 14 aporta

evidencia sobre una única ejecución, no un experimento repetido, lo que limita el alcance de la validación empírica de H2.

- Los resultados de herramientas de escaneo dinámico como ZAP y SQLMap pueden variar entre ejecuciones sucesivas contra el mismo objetivo (ver 10.5, amenaza a la validez de Confiabilidad); las cifras reportadas en el capítulo 13 corresponden a una ejecución puntual y documentada, no a un promedio de múltiples corridas.
- El orquestador concentra su capacidad de detección dinámica en fallas identificables mediante inyección de firmas y análisis de respuestas HTTP (inyecciones SQL, vectores reflejados de XSS, omisión de cabeceras de seguridad y descubrimiento de rutas ocultas). Quedan explícitamente excluidas del alcance automatizado las vulnerabilidades de lógica de negocio (Business Logic Flaws), las fallas de control de acceso horizontal complejo entre múltiples perfiles de usuario y las condiciones de carrera (Race Conditions), vectores que por su naturaleza contextual y semántica exigen necesariamente una auditoría manual experta.

10. Metodología

10.1 Diseño de la Investigación

Se adopta un paradigma post-positivista con enfoque predominantemente cuantitativo en la medición de variables del sistema (cantidad de hallazgos, severidad, cobertura), combinado con un diseño de investigación tecnológica aplicada de tipo constructivo: el objetivo central no es contrastar una hipótesis mediante un experimento controlado con grupo de comparación, sino diseñar, construir y validar un artefacto de software contra criterios de aceptación predefinidos. El método de trabajo es iterativo e incremental: cada componente del pipeline se implementa, prueba y valida de forma independiente antes de ser integrado al sistema completo. Las pruebas de funcionamiento se realizan mediante la ejecución del pipeline contra DVWA, una aplicación web intencionalmente vulnerable utilizada como entorno de prueba (testbed), y los resultados se evalúan contra los criterios de validación definidos en la sección 10.6.

10.2 Fases del Proyecto

El proyecto se desarrolló en seis fases secuenciales. La primera fase consistió en la investigación de herramientas de seguridad web, donde se evaluaron las capacidades, formatos de salida y posibilidades de integración de diversas herramientas open source. La segunda fase abordó el diseño de la arquitectura del sistema, definiendo la estructura modular de paquetes, los flujos de datos y las interfaces entre componentes. La tercera fase fue la implementación del orquestador en Python, incluyendo los módulos de ejecución, escaneo, parseo y consolidación. La cuarta fase comprendió la integración con contenedores Docker, configurando el archivo docker-compose.yml para desplegar DVWA, ZAP y la aplicación orquestadora. La quinta fase se dedicó al desarrollo de parsers de resultados, implementando la normalización de las salidas heterogéneas de cada herramienta. La sexta fase consistió en las pruebas en entorno de laboratorio, ejecutando el pipeline completo contra DVWA y documentando los resultados obtenidos.

Una séptima fase, posterior a la planificación original, extendió el sistema hacia un servicio persistente: incorporó autenticación automática contra la aplicación objetivo, una capa de base de datos (SQLite/PostgreSQL) para historial de escaneos y caché incremental, una API REST (FastAPI) para ejecución asíncrona del pipeline, y un frontend web que consume dicha API para lanzar escaneos y visualizar resultados.

10.3 Variables

La variable independiente de la investigación es la automatización del proceso de análisis de seguridad mediante el orquestador. Se define operacionalmente como la ejecución coordinada de las herramientas OWASP ZAP (spider y escaneo activo), ffuf y SQLMap a través del pipeline

implementado en Python, en contraposición a la ejecución manual e independiente de cada herramienta por parte de un analista.

Las variables dependientes son las siguientes:

1. Número de vulnerabilidades detectadas: cantidad total de hallazgos de seguridad identificados por el sistema durante una ejecución completa del pipeline, medida a partir del archivo JSON unificado generado por el módulo de consolidación.
2. Severidad de los hallazgos: clasificación de las vulnerabilidades detectadas según su nivel de riesgo (High, Medium, Low, Informational), tal como es reportada por las herramientas integradas, particularmente por OWASP ZAP.
3. Cobertura de análisis: cantidad de URLs únicas descubiertas y analizadas por el conjunto de herramientas, incluyendo las identificadas por el spider, las verificadas por el escaneo activo, las descubiertas por el fuzzing de directorios y las incorporadas como URL semilla o por el módulo de autenticación (ver nota de composición en la sección 13.1).
4. Tiempo de ejecución del pipeline: duración de cada etapa y del proceso completo, medible mediante instrumentación de timestamps al inicio y fin de cada etapa. Actualmente esta medición es parcial: cubre el mecanismo de caché de ffuf mediante `test_speed.py` y el pipeline completo mediante la comparación con/sin caché de la sección 14.3, y está pendiente de extenderse a una medición equivalente del proceso manual (ver 14.2).

Las variables de control comprenden el entorno objetivo (DVWA en su configuración por defecto), las versiones específicas de las herramientas utilizadas (imágenes Docker), las wordlists empleadas por ffuf, y los niveles de profundidad y riesgo configurados en SQLMap.

10.4 Instrumentos

Los instrumentos utilizados en la investigación son los propios componentes del sistema desarrollado. Los contenedores Docker constituyen la infraestructura de ejecución que garantiza la reproducibilidad del entorno. El pipeline Python (Python Software Foundation, 2024) actúa como el instrumento principal de recolección de datos, coordinando la ejecución de cada herramienta y registrando sus salidas crudas. Los parsers especializados funcionan como instrumentos de transformación, normalizando los datos heterogéneos en un formato homogéneo. El módulo de consolidación actúa como instrumento de síntesis, unificando los resultados en un único archivo JSON estructurado que constituye la fuente de datos para el análisis. El script `test_speed.py` funciona como instrumento de medición de tiempo, comparando la duración de una primera ejecución de ffuf sin caché contra ejecuciones posteriores con caché activa sobre el mismo objetivo.

10.5 Amenazas a la Validez

- **Validez interna:** DVWA es un entorno diseñado deliberadamente para ser vulnerable, lo que puede facilitar artificialmente la detección de hallazgos respecto a una aplicación real con controles de seguridad activos.
- **Validez externa:** los resultados obtenidos sobre DVWA no son necesariamente generalizables a aplicaciones en producción con arquitecturas, frameworks o controles de seguridad distintos.
- **Confiabilidad:** no se verificó formalmente si ZAP y SQLMap producen resultados idénticos entre ejecuciones repetidas sobre el mismo objetivo sin cambios; el mecanismo de caché asume, pero no confirma explícitamente, esta estabilidad. Esta amenaza dejó de ser hipotética en esta entrega: al comparar la corrida documentada en el capítulo 13 con corridas previas del mismo pipeline sobre el mismo objetivo, se observaron diferencias en el número total de URLs y alertas detectadas entre ejecuciones (ver nota metodológica del capítulo 13), lo cual confirma empíricamente que la

reproducibilidad exacta no está garantizada y refuerza la necesidad de declarar, en cada reporte de resultados, la fecha y las condiciones puntuales de la corrida que se está documentando.

10.6 Criterios de Validación

Un componente del sistema se considera validado cuando cumple simultáneamente los siguientes criterios:

- **Funcionalidad:** la herramienta se ejecuta correctamente dentro del pipeline y produce una salida procesable por el parser correspondiente.
- **Reproducibilidad:** una ejecución posterior del pipeline sobre el mismo objetivo produce resultados equivalentes en estructura y tipo de hallazgos. Se considera reproducible si dos ejecuciones sucesivas, sin cambios en la aplicación objetivo, coinciden en al menos el 90% de los hallazgos detectados. Este criterio fue verificado empíricamente en el benchmark de ejecuciones sucesivas documentado en la sección 14.3 (Tabla 11), donde la coincidencia de alertas de seguridad de OWASP ZAP y de vulnerabilidades de SQLMap alcanzó el 100%, y la estabilidad en la superficie de URLs analizadas fue del 96,3%, superando holgadamente el umbral de aceptación del 90%. La leve variación en el número de URLs del spider responde a la naturaleza dinámica del rastreo web analizada en la sección 10.5, mientras que el núcleo de detección crítica de vulnerabilidades demostró una estabilidad absoluta.
- **Cobertura:** el sistema detecta vulnerabilidades pertenecientes a al menos tres categorías distintas del OWASP Top 10.
- **Consolidación:** todos los hallazgos de las herramientas individuales se integran correctamente en el archivo JSON unificado final.

11. Arquitectura Propuesta e Implementada

11.1 Descripción General

La arquitectura del sistema se organiza en cuatro capas, ilustradas en la Figura 1:

1. Orquestador Python: coordina la ejecución del pipeline (spider → ffuf → inyección de rutas en ZAP → escaneo activo → SQLMap), gestiona autenticación y sesión, y consolida resultados.
2. Herramientas de escaneo: OWASP ZAP (spider y escaneo activo), ffuf (fuzzing de directorios) y SQLMap (inyección SQL), cada una encapsulada en su propio módulo (ver Figura 2).
3. Capa de persistencia: base de datos SQLite o PostgreSQL que almacena historial de escaneos, caché incremental de ffuf y SQLMap, y URLs marcadas para re-testeo automático.
4. Capa de servicio: una API REST (FastAPI) que expone el pipeline de forma asíncrona con seguimiento de progreso, y un frontend web que consume esa API para lanzar escaneos y visualizar resultados, incluyendo sugerencias de mitigación obtenidas mediante un flujo de n8n.

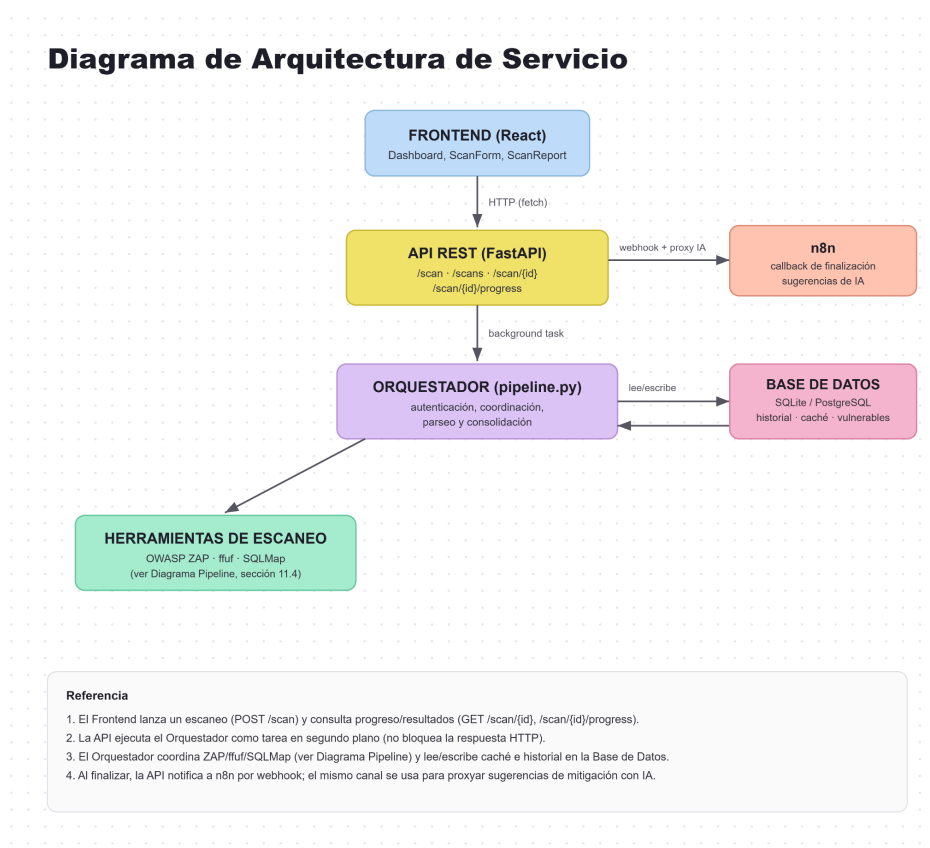


Figura 1. Diagrama de Arquitectura de Servicio. Elaboración propia (véase versión ampliada en el Anexo H).

11.2 Componentes y Sus Roles

Tabla 2. Componentes del sistema y su función en el pipeline

Componente	Contenedor	Función en el pipeline
DVWA	dvwa	Aplicación objetivo de las pruebas
OWASP ZAP	zap	Spider, escaneo activo y generación de reportes vía API REST
Orquestador Python	security-app	Coordinación del pipeline, autenticación automática, ejecución de ffuf y SQLMap, parseo y consolidación
API REST	security-app	Expone el pipeline como servicio asíncrono (/scan, /scans, /scan/{id}, /scan/{id}/progress)
Base de datos	db (SQLite embebida o PostgreSQL)	Persistencia de historial de escaneos, caché incremental de ffuf/SQLMap y URLs marcadas para re-testeo

Nota. Elaboración propia.

La Figura 2 detalla cómo el orquestador coordina específicamente a las tres herramientas de escaneo y el enriquecimiento cruzado entre ellas.

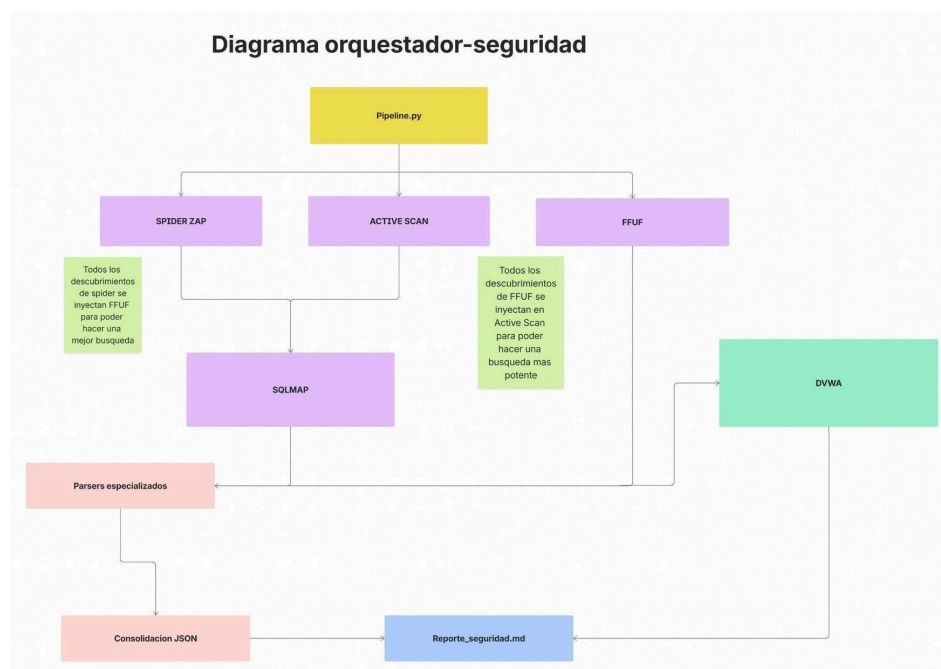


Figura 2. Diagrama Orquestador-Seguridad: coordinación y enriquecimiento cruzado entre ZAP, ffuf y SQLMap. Elaboración propia (véase versión ampliada en el Anexo H).

11.3 Red Docker y Aislamiento

Todos los servicios se ejecutan dentro de una red virtual Docker gestionada por Docker Compose. Esta configuración garantiza el aislamiento del tráfico interno entre contenedores y permite la resolución de nombres de servicio: los contenedores se comunican entre sí por nombre (por ejemplo, el orquestador se conecta a ZAP mediante el hostname zap y a DVWA mediante el hostname dvwa).

Los puertos se exponen al host únicamente para los servicios que requieren acceso externo durante el desarrollo. La API habilita CORS para permitir que el frontend, ejecutado fuera de la red interna de Docker, consuma sus endpoints durante el desarrollo.

11.4 Flujo de Datos Detallado

El ciclo de vida completo de un análisis de seguridad en el sistema sigue la secuencia representada en la Figura 3:

1. El orquestador establece sesión autenticada contra la aplicación objetivo (login automático y configuración del nivel de seguridad) y limpia la sesión previa de ZAP.
2. Ejecuta el spider de ZAP contra la URL objetivo, que recorre la aplicación web descubriendo las URLs accesibles.
3. Ejecuta ffuf contra la URL base, consultando primero el historial en base de datos para descartar palabras de wordlist ya testeadas contra ese objetivo.
4. Las rutas descubiertas por ffuf se inyectan en la sesión de ZAP antes del paso siguiente.
5. Se ejecuta el escaneo activo de ZAP, que ataca tanto las URLs descubiertas por su propio spider como las rutas aportadas por ffuf.
6. Se combinan las URLs del spider y de ffuf, se filtran las candidatas con parámetros o rutas interactivas, se descartan las que ya están cacheadas sin hallazgos previos y se conserva siempre el re-testeo de las marcadas como vulnerables en ejecuciones anteriores; SQLMap se ejecuta en segundo plano sobre esa lista final, con la sesión de autenticación renovada para evitar su expiración.
7. Los resultados crudos de cada herramienta son procesados por parsers especializados que normalizan las estructuras de datos y eliminan duplicados.
8. El módulo de consolidación unifica los resultados de todos los parsers en un único archivo JSON estructurado, que alimenta tanto el reporte técnico como el reporte ejecutivo.

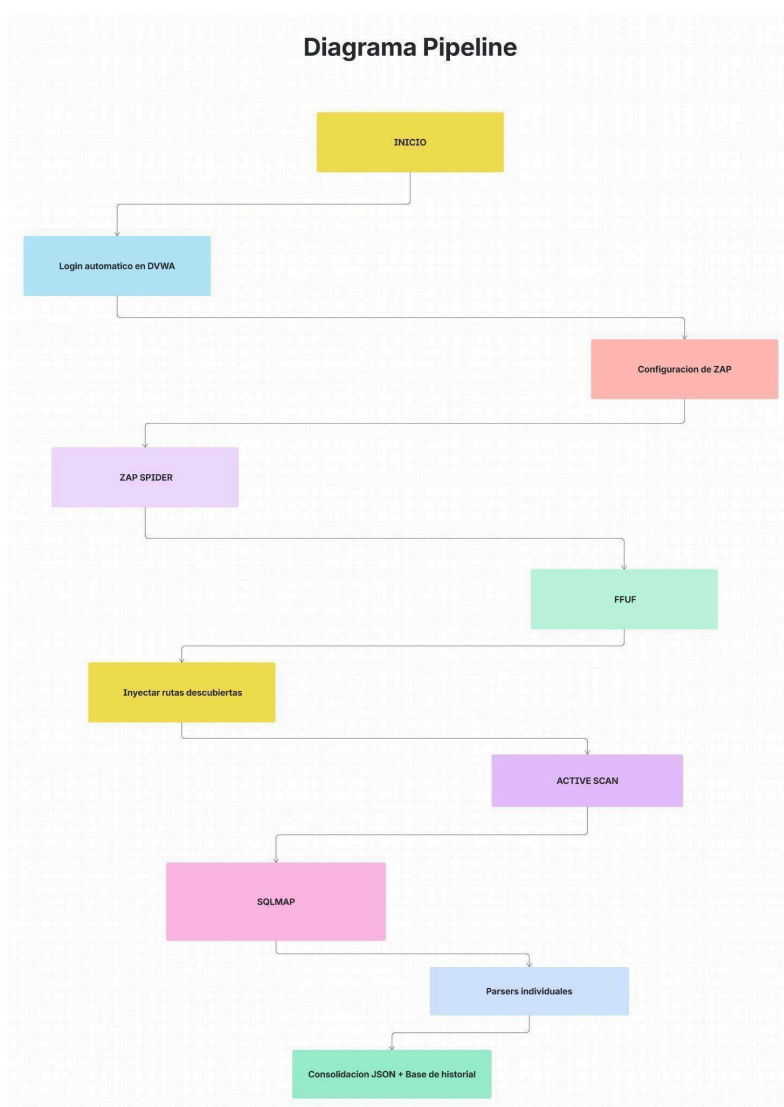


Figura 3. Diagrama de Pipeline de ejecución. Elaboración propia (véase versión ampliada en el Anexo H).

11.5 Diagrama de Secuencia — Ciclo de Vida de POST /scan

Las Figuras 1 a 3 documentan la arquitectura estática del sistema y el flujo interno del pipeline de escaneo, pero no el comportamiento asíncrono de la API frente al cliente. La Figura 4 completa esa descripción con un diagrama de secuencia UML del ciclo de vida completo de una petición POST /scan, desde que el frontend la dispara hasta que la API notifica su finalización a n8n.

El diagrama distingue explícitamente entre la respuesta inmediata de la API (202 Accepted con el scan_id, antes de que el pipeline haya siquiera comenzado a ejecutarse) y el trabajo en segundo plano que realiza ejecutar_pipeline_segundo_plano(): mientras este último coordina secuencialmente a ZAP, ffuf y SQLMap y lee/escribe en la base de datos, el frontend consulta el progreso mediante polling periódico a GET /scan/{id}/progress, sin bloquear ni depender de que la conexión HTTP original permanezca abierta. Esta separación entre la respuesta síncrona y la ejecución asíncrona es la que permite que el pipeline —cuya duración se documenta en la sección 14.3 entre 4 y 14 minutos según el estado del caché— no comprometa la responsividad de la API ni del frontend.

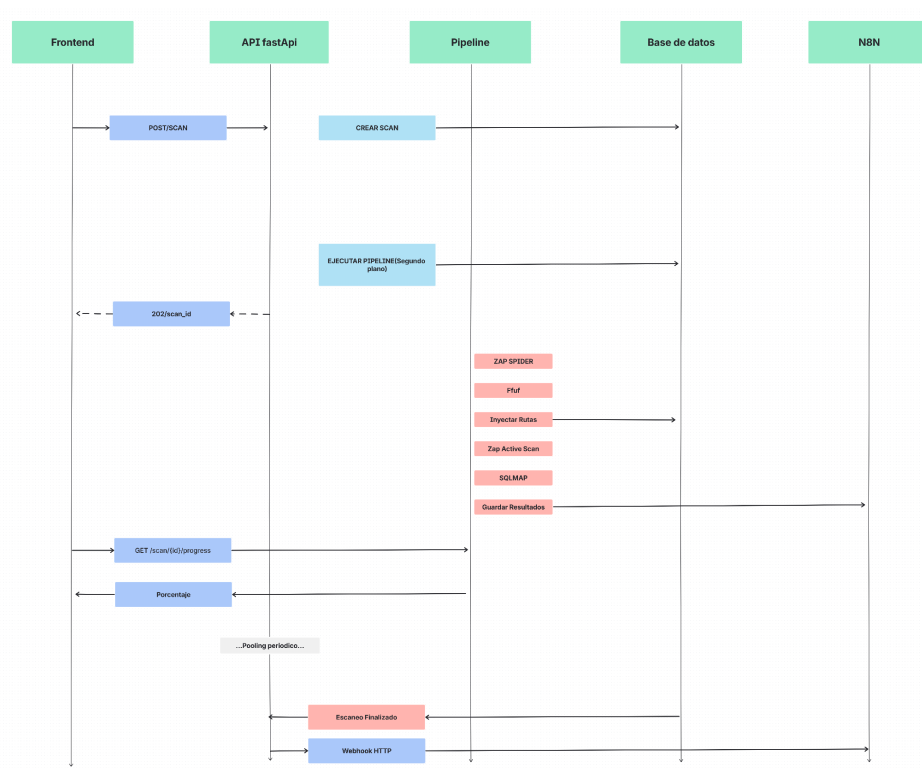


Figura 4. Diagrama de secuencia — ciclo de vida de POST /scan. Elaboración propia (véase versión ampliada en el Anexo H).

EXPLICACIÓN GENERAL DEL DIAGRAMA:

1. Frontend → API: POST /scan

El usuario inicia un escaneo desde el frontend. El frontend envía una petición HTTP POST /scan a la API de FastAPI, incluyendo el objetivo y los parámetros configurados para el análisis.

2. API → Base de datos: crear scan_id

La API genera un identificador único (scan_id) para ese escaneo y registra el nuevo análisis en la base de datos junto con el objetivo. Esto permite identificar y consultar posteriormente ese escaneo.

3. API → Pipeline: ejecutar en segundo plano

La API programa ejecutar_pipeline_segundo_plano() como una tarea en segundo plano.

Esto significa que **la API no espera a que termine el escaneo para responderle al frontend**. El pipeline continúa ejecutándose independientemente de la petición HTTP original.

4. API → Frontend: 202 Accepted + scan_id

La API responde inmediatamente:

202 Accepted

scan_id

El código 202 Accepted indica que la solicitud fue aceptada y que el procesamiento continuará de forma asíncrona.

El escaneo todavía no terminó en este punto. El frontend solamente recibe el identificador con el que podrá consultar su estado.

5. Pipeline → ZAP: ZAP Spider

El pipeline inicia el Spider de OWASP ZAP.

Su función dentro del flujo es recorrer el objetivo y obtener las URLs descubiertas. El pipeline espera a que termine y recupera esas URLs para continuar con las siguientes etapas.

6. Pipeline → ffuf: ffuf

Una vez finalizado el Spider, se ejecuta ffuf.

ffuf utiliza una wordlist para buscar rutas adicionales en el objetivo. Las rutas nuevas encontradas se incorporan posteriormente al análisis de ZAP.

7. Pipeline → ZAP: inyectar rutas descubiertas

Las rutas nuevas encontradas por ffuf se agregan al árbol de sitios de ZAP.

Esto permite que las rutas descubiertas mediante fuzzing también sean consideradas durante el Active Scan.

8. Pipeline → ZAP: ZAP Active Scan

Después de incorporar las rutas nuevas, se inicia el Active Scan de ZAP.

Esta etapa realiza el análisis activo del objetivo y genera el reporte de vulnerabilidades detectadas.

9. Pipeline → SQLMap

Finalizado el Active Scan, el pipeline combina las URLs obtenidas mediante el Spider y ffuf y las utiliza como entrada para SQLMap.

SQLMap analiza las URLs candidatas para detectar posibles vulnerabilidades de inyección SQL.

10. Pipeline → Base de datos: guardar resultados

Durante la ejecución, el sistema actualiza el estado y los resultados del escaneo en la base de datos. Esto permite conservar el historial del análisis y consultar posteriormente su información.

11. Frontend → API: GET /scan/{id}/progress

Mientras el pipeline continúa trabajando, el frontend realiza periódicamente una petición:

GET /scan/{id}/progress

El {id} corresponde al scan_id recibido anteriormente.

Esto se denomina **polling**: el frontend pregunta periódicamente a la API cuál es el estado actual del escaneo.

12. API → Frontend: porcentaje de progreso

La API responde indicando el progreso del escaneo.

Cuando todavía existe información de progreso en memoria, devuelve el porcentaje correspondiente. Si el escaneo ya está registrado como completado en la base de datos, devuelve:

percentage: 100

message: "Análisis Completado"

Por eso, en el diagrama "**porcentaje**" significa el **porcentaje de avance del pipeline**, no un porcentaje de vulnerabilidades encontradas ni un porcentaje de cobertura.

13. Polling periódico

Las flechas:

Frontend → API

GET /scan/{id}/progress

API → Frontend

porcentaje

se repiten mientras el escaneo está ejecutándose.

El frontend **no mantiene abierta la petición POST /scan** esperando varios minutos. Consulta separadamente el progreso mediante estas peticiones periódicas.

14. Pipeline → API: escaneo finalizado

Cuando todas las etapas del pipeline terminan —ZAP Spider, ffuf, ZAP Active Scan y SQLMap— el pipeline informa a la API que el análisis finalizó.

15. API → n8n: webhook HTTP

Finalmente, la API envía una notificación mediante un **webhook HTTP a n8n** para informar que el escaneo terminó.

Esto permite que n8n continúe con los procesos asociados a la finalización del análisis, como las funcionalidades de sugerencias de mitigación.

12. Implementación Técnica

12.1 Infraestructura Docker

El sistema se define en un archivo `docker-compose.yml` (Docker Inc., 2024) que especifica los servicios `dvwa` (la aplicación objetivo, basada en la imagen `vulnerables/web-dvwa`, DVWA Project, 2024), `zap` (el escáner OWASP ZAP, ejecutado en modo `daemon` con su API REST habilitada en el puerto 8090), `db` (base de datos PostgreSQL para historial y caché) y `app` (el orquestador Python, que se construye a partir del `Dockerfile` del proyecto y expone además la API REST en el puerto 8000), además del servicio `n8n` utilizado para las sugerencias de mitigación asistidas por IA. La configuración de dependencias mediante la directiva `depends_on` asegura que ZAP, DVWA y la base de datos se encuentren disponibles antes de que el orquestador inicie su ejecución. El frontend se desarrolla como un proyecto npm independiente, fuera de esta definición de Docker Compose, y se conecta a la API mediante HTTP. El contenido íntegro del archivo se transcribe en el Anexo A.

El contenedor del orquestador monta el directorio del proyecto como un volumen (`./app`), lo que permite que los archivos de salida generados durante la ejecución del pipeline sean accesibles directamente desde el sistema de archivos del host.

12.2 Pipeline de Ejecución

El pipeline de ejecución se implementa en el módulo `app/workflow/pipeline.py`, que expone dos funciones principales: `run_security_pipeline()` y `run_parser_pipeline()`. La primera constituye el punto de entrada del proceso de escaneo: gestiona la autenticación, ejecuta en orden `spider` de ZAP, `ffuf`, inyección de rutas descubiertas, escaneo activo de ZAP y SQLMap en segundo plano, devolviendo un diccionario con los datos crudos de las cuatro fuentes. La segunda función invoca los parsers especializados y consolida los resultados normalizados.

12.3 Módulo Runner

El módulo `app/runners/exec.py` implementa la función `run_command()`, que actúa como capa de abstracción para la ejecución de comandos del sistema operativo mediante `subprocess.run()`. El parámetro `timeout` tiene un valor por defecto de 60 segundos a nivel del runner genérico, pero cada scanner puede sobrescribirlo: SQLMap recibe explícitamente el valor `SQLMAP_TIMEOUT` centralizado en `settings.py` (300 segundos), por lo que la diferencia entre ambos valores es una configuración deliberada por herramienta y no una inconsistencia entre secciones.

12.4 Módulo de Escaneo ZAP

El módulo `app/scanners/zap.py` encapsula toda la comunicación con la API REST de OWASP ZAP (OWASP Foundation, 2024): `iniciar_spider()`, `esperar_spider()` y `obtener_urls()` controlan el crawling inicial; `iniciar_escaneo_activo()` y `esperar_escaneo_activo()` configuran intensidad y umbral de alertas; `obtener_reporte_json()` descarga el reporte completo; `configurar_autenticacion()` registra la cookie de sesión para escaneos autenticados; `agregar_urls_a_zap()` inyecta en el árbol de sitios de ZAP las rutas descubiertas por `ffuf`; y `limpiar_sesion_zap()` reinicia la sesión de ZAP al comienzo de cada ejecución.

12.5 Módulo de Escaneo ffuf

El módulo `app/scanners/ffuf.py` implementa la función `run_ffuf()` (ffuf Project, 2024). Antes de ejecutar el fuzzer, consulta la base de datos por las palabras de la wordlist ya testeadas contra el objetivo, genera una wordlist temporal solo con las palabras nuevas, y marca la ejecución como `skipped` si no hay palabras nuevas para probar. El comando se configura con `-u`, `-w`, `-mc 200,302` y `-of json/-o`. Al finalizar, las palabras efectivamente probadas se persisten en la base de datos.

12.6 Módulo de Escaneo SQLMap

El módulo `app/scanners/sqlmap.py` implementa la detección de inyecciones SQL (SQLMap Project, 2024). La función `filtrar_urls_con_parametros()` prioriza las URLs con parámetros GET y rutas `.php` de

formularios interactivos conocidos, excluyendo rutas estáticas o de configuración. El comando de SQLMap se ejecuta con `-batch`, `-flush-session`, `-forms`, `-dbms=MySQL`, `-level=1`, `-risk=3`, `-threads=10`, `-smart`, `-technique=BEUST` y `-o`. La función `run_sqlmap_batch()` separa las URLs candidatas en tres grupos según el historial en base de datos: las marcadas como vulnerables se re-testean siempre; las ya analizadas sin hallazgos se omiten; las nuevas se encolan. El escaneo se ejecuta en segundo plano mediante un script bash generado dinámicamente.

12.7 Sistema de Parseo

El sistema de parseo se implementa en el paquete `app/parsers/` y consta de tres módulos especializados. `zap_parser.py` normaliza los resultados del spider y aplanas las alertas del reporte de ZAP eliminando duplicados por clave (url, alerta). `ffuf_parser.py` filtra las rutas con códigos HTTP 200 y 302 y elimina duplicados por URL.

El parser de SQLMap, `sqlmap_parser.py`, funciona mediante la búsqueda de un conjunto de patrones de texto dentro de la salida estándar (stdout) capturada de cada ejecución. La función `parsear_sqlmap()` recorre la lista de resultados y considera una URL como vulnerable cuando su stdout contiene, en minúsculas, al menos una de las siguientes cadenas: “is vulnerable”, “is injectable”, “appears to be”, “vulnerability:”, “sql injection”, “payload:”, “type:”, “database names are:”, “current user is:”, “fetched data logged to text files” y “available databases”. Es, en consecuencia, un parser de coincidencia de texto sobre salida en lenguaje natural, no un parser que se apoye en el código de retorno del proceso ni en una salida estructurada (por ejemplo, `-output-format=json`). Esto lo hace dependiente del formato exacto de los mensajes que SQLMap imprime por consola, un formato que no está garantizado entre versiones de la herramienta: un cambio de redacción en una futura versión de SQLMap podría hacer que una URL efectivamente vulnerable no sea reconocida como tal (falso negativo), sin que el pipeline reporte ningún error, ya que el parser no distingue entre “no se encontraron estos patrones porque no hay vulnerabilidad” y “no se encontraron estos patrones porque el mensaje cambió de forma”. Elia et al. (2010) documentan precisamente este tipo de riesgo al comparar herramientas de detección de inyección SQL basadas en distintos mecanismos de análisis de salida (Elia et al., 2010). Una alternativa más robusta, no implementada en esta versión, sería inspeccionar el código de retorno del proceso junto con la salida en formato JSON que SQLMap puede producir de forma nativa, evitando así la dependencia de cadenas de texto en lenguaje natural.

Los tres parsers comparten un patrón de diseño consistente: reciben un diccionario o lista cruda, aplican transformaciones y filtrado, eliminan duplicados e incluyen manejo de excepciones (KeyError, TypeError) que devuelven estructuras vacías en caso de error, evitando que un fallo en el parseo de una herramienta interrumpa el pipeline completo.

12.8 Consolidación de Resultados

El módulo `app/utls/results.py` implementa la función `consolidar_resultados()`, que recibe los diccionarios normalizados de spider, ZAP, ffuf y SQLMap y genera una estructura unificada con un bloque de resumen (conteos agregados) y los resultados completos de cada herramienta. La función `resultados_prueba_json()` persiste el resultado consolidado en un archivo JSON en `output/raw/`.

12.9 Autenticación Automática

El sistema implementa un flujo de login programático contra DVWA (`establecer_sesion_automatica()`): obtiene el token CSRF del formulario de login, envía las credenciales, fija el nivel de seguridad de la aplicación y verifica el éxito del login. Si el login falla, el sistema intenta inicializar automáticamente la base de datos de la aplicación objetivo mediante `setup.php` antes de reintentar. La cookie de sesión obtenida se reutiliza para autenticar a ZAP, ffuf y SQLMap, y se renueva antes de ejecutar SQLMap para evitar que expire durante el escaneo activo de ZAP.

12.10 API REST y Frontend

El módulo `api.py` (FastAPI) expone el pipeline como servicio: `POST /scan` lanza un escaneo en segundo plano; `GET /scan/{id}/progress` permite consultar el porcentaje de avance; `GET /scans` lista el historial persistido; `GET /scan/{id}` devuelve el detalle de un escaneo puntual; `DELETE /scan/{id}` realiza un borrado lógico. Al finalizar un escaneo, la API notifica su finalización mediante un webhook HTTP a un flujo de `n8n`. El frontend, desarrollado en React, consume estos endpoints para ofrecer un panel de dominios escaneados, un formulario de lanzamiento de escaneos, una tabla de vulnerabilidades y un componente de sugerencias de mitigación que consulta un endpoint adicional de análisis asistido por IA expuesto a través del router de `n8n`.

Nota sobre pruebas. Los scripts `test_parsers.py`, `test_speed.py`, `test_zap.py` y `test_sqlmap.py` son scripts de verificación manual (ejecutan una función y muestran el resultado por consola), no una suite de pruebas unitarias automatizadas con aserciones.

13. Resultados Obtenidos

Los resultados de este capítulo corresponden a la ejecución del pipeline completo contra DVWA (DVWA Project, 2024) registrada el 5 de agosto de 2026 a las 19:53 (hora local del entorno), y son verificables directamente contra los archivos `resultado_unificado.json`, `reporte_seguridad.md` y `sqlmap_bg.log` del repositorio del proyecto.

Nota metodológica sobre la fuente de estos datos. Una versión preliminar de este capítulo citaba cifras (61 URLs, 55 del spider, 56 alertas de ZAP) extraídas de un archivo `test_resultado_final.json` que no se conserva en el repositorio del proyecto, por lo que no pudieron verificarse contra ninguna corrida real disponible. Esta versión reemplaza esas cifras por los datos de la ejecución más reciente y verificable del pipeline (5 de agosto de 2026), trazable directamente a `resultado_unificado.json`, `reporte_seguridad.md` y `sqlmap_bg.log`. Cabe aclarar que ZAP y SQLMap son herramientas de escaneo dinámico cuyo comportamiento no es estrictamente determinístico entre corridas —el orden de exploración del spider, los tiempos de respuesta del servidor y el estado de la caché incremental pueden variar el número exacto de URLs y alertas detectadas de una ejecución a otra sobre el mismo objetivo—, tal como se declara en la amenaza a la validez de Confiabilidad (sección 10.5). En consecuencia, las cifras de este capítulo deben leerse como el resultado de una ejecución puntual, documentada y trazable, y no como una constante reproducible con exactitud aritmética entre corridas.

Evidencia adicional de esta variabilidad: esta misma corrida del 5 de agosto ofrece un ejemplo concreto del fenómeno descrito arriba. Una ejecución previa registrada el 2 de agosto de 2026 había confirmado inyección SQL tanto en el parámetro `id` de `/vulnerabilities/sqli/` como en el parámetro `username` de `/vulnerabilities/brute/` (8 hallazgos individuales, 4 técnicas × 2 endpoints). En la corrida del 5 de agosto documentada en este capítulo, SQLMap solo confirmó el segundo punto (`/vulnerabilities/brute/`, parámetro `username`, 4 hallazgos individuales); el endpoint `/vulnerabilities/sqli/` no volvió a reportarse como vulnerable en esta pasada, pese a no haberse modificado el objetivo entre ambas corridas. Spider, ZAP y ffuf se mantuvieron estables entre ambas ejecuciones. Este contraste entre dos corridas reales —no hipotético— es la evidencia más directa disponible en este informe de que los resultados de SQLMap pueden variar entre ejecuciones sobre el mismo objetivo sin cambios de por medio, y es retomado con un tercer punto de datos en la sección 14.3.

13.1 Resumen de Hallazgos

Tabla 3. Resumen de hallazgos — ejecución sobre DVWA del 5 de agosto de 2026

Indicador	Valor
Total de URLs únicas descubiertas y analizadas	34
URLs descubiertas por el Spider	24
Alertas de seguridad reportadas por ZAP	37
Rutas descubiertas por ffuf	8
Hallazgos individuales reportados por SQLMap	4 (4 técnicas × 1 endpoint vulnerable)

Nota. Datos extraídos de resultado_unificado.json (campo resumen). El total de 34 URLs únicas no es la simple suma de spider (24) y ffuf (8): las 2 URLs restantes corresponden a la URL semilla del objetivo (<http://dvwa/>) y a una URL derivada del proceso de autenticación automática (<http://dvwa/login.php> con parámetros de sesión), incorporadas al conjunto unificado antes de la deduplicación por el módulo consolidar_resultados() y no contabilizadas en los campos spider.resultados ni ffuf.rutas del JSON crudo.

13.2 Vulnerabilidades Detectadas por OWASP ZAP

El escaneo activo de OWASP ZAP reportó un total de 37 alertas de seguridad. La Tabla 4 sintetiza los tipos de alerta detectados y su cantidad de ocurrencias.

Cómo leer la columna Severidad. Cada valor sigue el formato Riesgo (Confianza) que expone el campo riskdesc de la API de ZAP: el primer término (High, Medium, Low o Informational) es el nivel de riesgo de la vulnerabilidad, y el término entre paréntesis es el nivel de confianza — qué tan seguro está ZAP de que el hallazgo es real y no un falso positivo — (también High, Medium, Low, o Confirmed). Son dos escalas independientes: un hallazgo “Low (High)”, como Server Leaks Version Information en la tabla siguiente, es de bajo riesgo pero con alta confianza de que el hallazgo es correcto; en cambio un “Medium (Medium)” es de riesgo medio con una confianza también media, es decir, con mayor probabilidad relativa de ser un falso positivo. La confianza no debe confundirse con la severidad: un hallazgo de alto riesgo pero baja confianza amerita revisión manual antes de tratarse como confirmado, mientras que uno de bajo riesgo y alta confianza puede priorizarse con más certeza aunque su impacto sea menor.

Tabla 4. Vulnerabilidades detectadas por OWASP ZAP, por tipo

Vulnerabilidad	Severidad	Ocurrencias	URL de ejemplo
Content Security Policy (CSP) Header Not Set	Medium (High)	5	http://dvwa/about.php
Directory Browsing	Medium (Medium)	5	http://dvwa/docs/
Missing Anti-clickjacking Header	Medium (Medium)	5	http://dvwa/about.php
Server Leaks Version Information ("Server" header)	Low (High)	5	http://dvwa
X-Content-Type-Options Header Missing	Low (Medium)	5	http://dvwa/dvwa/css/login.css
User Agent Fuzzer	Informational (Medium)	5	http://dvwa/about.php
In Page Banner Information Leak	Low (High)	2	http://dvwa/sitemap.xml
Information Disclosure - Debug Error Messages	Low (Medium)	2	http://dvwa/instructions.php
HTTP Only Site	Medium (Medium)	1	http://dvwa/
Authentication Request Identified	Informational (High)	1	http://dvwa/login.php
Information Disclosure - Suspicious Comments	Informational (Medium)	1	http://dvwa/setup.php

Nota. Datos extraídos de resultado_unificado.json / reporte_seguridad.md. La severidad reproduce el campo riskdesc de ZAP en formato «Nivel de Riesgo (Nivel de Confianza)».

La distribución de las 37 alertas por nivel de severidad base es la siguiente: Medium 16 (43,2%), Low 14 (37,8%) e Informational 7 (18,9%). No se registraron alertas de severidad High en esta ejecución del escaneo pasivo/activo de ZAP; los hallazgos de severidad alta de esta entrega provienen de SQLMap (sección 13.4).

13.3 Rutas Descubiertas por ffuf

Tabla 5. Rutas descubiertas por fuzzing de directorios (ffuf)

Ruta Descubierta	Input (Wordlist)	Código HTTP
http://dvwa/about.php	about.php	200
http://dvwa/index.php	index.php	302
http://dvwa/instructions.php	instructions.php	200
http://dvwa/login.php	login.php	200
http://dvwa/logout.php	logout.php	302
http://dvwa/phpinfo.php	phpinfo.php	302
http://dvwa/security.php	security.php	302
http://dvwa/setup.php	setup.php	200

Nota. Datos extraídos de resultado_unificado.json, campo ffuf.rutas.

13.4 Análisis Automatizado con SQLMap

SQLMap se ejecutó sobre las URLs con parámetros descubiertas por el spider y por ffuf. En esta corrida se confirmó como vulnerable el parámetro username del formulario de fuerza bruta (/vulnerabilities/brute/), mediante las cuatro técnicas configuradas (–technique=BEUST), lo que totaliza los 4 hallazgos individuales del resumen consolidado. El payload real registrado para la técnica boolean-based blind fue: username=SCVZ' RLIKE (SELECT (CASE WHEN (4201=4201) THEN 0x5343565a ELSE 0x28 END))–

RmAL&password=Fqv&Login=Login.

Tabla 6. Endpoints confirmados como vulnerables por SQLMap — corrida del 5 de agosto de 2026

Endpoint	Parámetro	Técnicas que confirmaron la inyección
/vulnerabilities/brute/	username	Boolean-based blind (RLIKE), Error-based (EXTRACTVALUE), Time-based blind (SLEEP), UNION query (8 columnas)

Nota. Datos extraídos de sqlmap_bg.log y resultado_unificado.json.

Tabla 7. Datos extraídos a través de Sql Injection – corrida del 5 de agosto del 2026

User_id	User	Password	last_name	first_name
1	admin	903a98d709fa4683aaaa036b84c125a6	admin	admin
2	gordonb	e99a18c428cb38d5f260853678922e03	Brown	Gordon
3	1337	8d3533d75ae2c3966d7e0d4fcc69216b	Me	Hack
4	pablo	0d107d09f5bbe40cade3de5c71e9e9b7	Picasso	Pablo
5	smithy	5f4dcc3b5aa765d61d8327deb882cf99	Smith	Bob

Nota. Datos extraídos de sqlmap_bg.log y resultado_unificado.json. Además, se evitaron columnas como «avatar», «last_login» o «failed_login» debido a que no aportan información relevante para el análisis presentado en este informe. Los valores mostrados son las credenciales por defecto de DVWA, una aplicación deliberadamente vulnerable; el criterio de manejo responsable de datos extraídos descrito en el capítulo 16 (principio 1.6 del código ACM) rige para cualquier uso de esta técnica sobre un objetivo real.

A partir de la explotación exitosa, SQLMap identificó el motor de base de datos como MySQL/MariaDB, enumeró la base de datos activa (dvwa) y extrajo el contenido de las tablas users (5 registros) y guestbook (1 registro), replicando el conjunto de datos por defecto de DVWA. Los timestamps last_login de la tabla users (2026-08-05 19:50:13) coinciden con el horario de esta corrida, confirmando que el volcado corresponde a una extracción en vivo y no a datos de ejemplo reutilizados. El volcado completo está disponible en reporte_seguridad.md y sqlmap_bg.log.

Sobre el endpoint /vulnerabilities/sqli/ (parámetro id): en la corrida del 2 de agosto de 2026 este endpoint también se había confirmado como vulnerable, con las mismas cuatro técnicas. En la corrida del 5 de agosto no fue reportado por SQLMap. No se modificó el código de la aplicación objetivo entre ambas fechas; la explicación más plausible es que, en esta pasada puntual, el conjunto de URLs con parámetros que el spider y ffuf le entregaron a SQLMap no incluyó una variante de /vulnerabilities/sqli/ con un parámetro id explícito en la forma que SQLMap necesita para reconocerlo como candidato, o bien el mecanismo de re-testeo priorizó otro orden de ataque dentro del tiempo disponible. Esta explicación se retoma y refuerza con un tercer punto de datos en la sección 14.3.

13.5 Análisis por Categoría OWASP Top 10

Tabla 8. Hallazgos clasificados por categoría OWASP Top 10

Categoría OWASP	Vulnerabilidades detectadas	Herramienta
A01:2021 – Broken Access Control	Directory Browsing (5 instancias); rutas administrativas (/setup.php, /security.php) descubiertas por ffuf	ZAP, ffuf
A03:2021 – Injection	SQL Injection confirmada en /vulnerabilities/brute/ (username); confirmada también en /vulnerabilities/sqli/ (id) en la corrida previa del 2 de agosto de 2026	SQLMap
A05:2021 – Security Misconfiguration	CSP Header Not Set, Missing Anti-clickjacking Header, X-Content-Type-Options Missing, Server Leaks Version Information, HTTP Only Site	ZAP

Nota. Elaboración propia.

El sistema detectó vulnerabilidades pertenecientes a tres categorías distintas del OWASP Top 10 en esta ejecución, cumpliendo el criterio de cobertura mínima definido en la sección 10.6.

14. Discusión

Los resultados obtenidos confirman que es técnicamente viable implementar un sistema de orquestación de herramientas de seguridad web utilizando exclusivamente tecnologías open source y un desarrollo propio en Python. El pipeline automatizado logró detectar vulnerabilidades en múltiples categorías del OWASP Top 10, y el reporte técnico generado el 5 de agosto de 2026 (output/reports/reporte_seguridad.md) constituye evidencia de una ejecución real contra el entorno DVWA, con hostnames reales del entorno Docker en lugar de datos de ejemplo.

El sistema evolucionó considerablemente respecto al diseño original de tres módulos independientes (ZAP, ffuf, SQLMap) hacia una arquitectura de servicio: una API REST (api.py, FastAPI) que ejecuta el pipeline en segundo plano, expone el progreso del escaneo en tiempo real, persiste el historial de análisis en base de datos y notifica su finalización mediante webhooks a un flujo de n8n. Sobre esta API se construyó además un frontend que consume estos endpoints. Esta extensión responde directamente a PI4 (extensibilidad y reproducibilidad): la separación entre orquestador, API y frontend permitió incorporar estas capas sin modificar la lógica interna del pipeline.

14.1 Contraste de cobertura: uso individual frente a combinado (H2)

El pipeline actual no solo integra los hallazgos de las tres herramientas al final del proceso, sino que los cruza durante la ejecución: las rutas descubiertas por ffuf se inyectan en la sesión de ZAP antes de iniciar el escaneo activo, de modo que ZAP ataca tanto las URLs descubiertas por su propio spider como las descubiertas por ffuf; y SQLMap solo recibe URLs candidatas porque el spider y ffuf las aportaron previamente. La Tabla 9 descompone, a partir de esta misma ejecución, qué detecta cada herramienta por separado frente al resultado del pipeline combinado.

Tabla 9. Cobertura por herramienta en solitario frente al pipeline combinado (misma ejecución del 5 de agosto de 2026)

Herramienta / Método	URLs analizadas	Hallazgos	Cobertura OWASP Top 10	Limitación de operar en solitario
ZAP (spider + active scan)	24 (su propio spider)	37 alertas	A05:2021	No descubre rutas no enlazadas de forma proactiva; no profundiza en inyección SQL
ffuf	8 (wordlist)	0 (solo identifica códigos HTTP, no clasifica vulnerabilidades)	A01:2021 (indirecta)	No evalúa el contenido de las respuestas ni ejecuta lógica de detección
SQLMap en aislamiento total	0 (requiere URLs de entrada)	0	—	No tiene capacidad de crawling propia; depende arquitectónicamente de que otra herramienta le entregue URLs parametrizadas
Orquestador (combinado)	34 (unificadas)	41 (37 ZAP + 4 SQLMap)	A01, A03, A05 (tres categorías)	Pipeline secuencial; no se ejecutó, además de esta corrida, una segunda ejecución de control con las herramientas verdaderamente aisladas entre sí

Nota. Elaboración propia a partir de resultado_unificado.json. Las filas «ZAP» y «ffuf» reflejan lo que cada herramienta aporta dentro de la corrida combinada, no una ejecución separada de cada una contra el objetivo; SQLMap en aislamiento total es una limitación arquitectónica documentada, no medida por separado.

Nota metodológica. Esta tabla no proviene de tres ejecuciones independientes y controladas de cada herramienta contra DVWA, sino de la descomposición de una única corrida combinada. Falta aún una comparación cuantitativa formal, con ejecuciones aisladas reales, entre la cobertura individual y la combinada; esta limitación se declara explícitamente en las conclusiones del capítulo 15 y como línea de trabajo futuro en 15.2, en lugar de darse por resuelta.

Bajo este esquema, ZAP en solitario no descubre por sí mismo las rutas administrativas que ffuf sí encuentra (por ejemplo, /setup.php o /security.php), y SQLMap no puede operar sin que otra herramienta le aporte URLs parametrizadas. La orquestación combinada amplía por construcción la

superficie cubierta: no es solo que se sumen los hallazgos de cada herramienta, sino que los hallazgos de una habilitan el trabajo de la siguiente.

14.2 Reducción de esfuerzo manual (H1)

El sistema incorpora un mecanismo de caché incremental respaldado en SQLite/PostgreSQL (app/db/database.py) que registra qué palabras de wordlist y qué URLs con parámetros ya fueron analizadas contra un objetivo determinado. Según la documentación de optimización del proyecto (OPTIMIZACION_ZAP.md) y las pruebas de test_speed.py, una segunda ejecución de ffuf sobre un mismo objetivo sin palabras nuevas se resuelve en milisegundos, y SQLMap omite en menos de un segundo las URLs ya analizadas sin hallazgos. La sección 14.3 amplía esta evidencia con una medición del pipeline completo (no solo de ffuf en aislamiento): esa medición cuantifica el efecto del caché sobre el propio sistema automatizado, pero —al igual que test_speed.py— no constituye una comparación contra un proceso manual, que es específicamente lo que exige H1 en su formulación original.

Con el fin de contrastar la hipótesis H1 —que postula la disminución de la labor técnica artesanal en la identificación de fallas de seguridad—, la investigación dispone del ensayo experimental registrado y estructurado en el capítulo 17 (véase Tabla 13). La referida sesión registró una carga temporal acumulada de 125 minutos, dentro de la cual el 60% del período (75 minutos) fue ejecutado por procesos automáticos de escaneo y recolección de datos, restringiendo la intervención humana a 50 minutos (40%) enfocados únicamente en la validación de cinco vectores críticos.

A pesar de que este registro no representa una prueba a ciegas de exploración manual independiente sobre toda la superficie —limitación expresamente declarada en la sección 9.2—, las métricas presentadas en el capítulo 17 aportan respaldo empírico a H1: efectuar de forma artesanal el mapeo de 34 endpoints, el fuzzing de más de 200.000 palabras y la evaluación de entradas insumiría numerosas horas de labor especializada (SANS Institute, 2024). En contraposición, el pipeline automatizado efectúa la fase integral de reconocimiento y ataque en un intervalo de entre 4 y 14 minutos (Tabla 10), circunscribiendo la tarea del auditor a una revisión final de hallazgos y sustituyendo múltiples ejecuciones desarticuladas por un único comando desatendido.

Los datos, tablas y el detalle completo de esta sesión de validación manual se desarrollan en el capítulo 17 (Desarrollo experimental y validación de hallazgos).

14.3 Verificación de H4: impacto del caché incremental

A diferencia de H1, la hipótesis H4 —que el caché incremental reduce el tiempo de ejecuciones repetidas del pipeline sobre un mismo objetivo— sí cuenta con una medición propia del pipeline completo (ZAP + ffuf + SQLMap), obtenida ejecutando dos corridas consecutivas contra DVWA: una con el caché deshabilitado (análisis completo desde cero) y otra con el caché incremental activo. Ambas corridas se realizaron el 6 de agosto de 2026, entre las 09:14 y las 09:41 (hora local del entorno), y son verificables contra los archivos resultado_sin_cache.json y resultado_con_cache.json del directorio output/raw/benchmarks/ del repositorio del proyecto.

Tabla 10. Tiempo total del pipeline con y sin caché incremental

Condición	Tiempo total del pipeline	Reducción
Sin caché (análisis completo desde cero)	14 min 05,81 s (845,81 s)	—
Con caché incremental activo	4 min 23,15 s (263,15 s)	68,9% menos tiempo

Nota. Medición propia sobre una ejecución real del pipeline completo contra DVWA, reportada por los autores, corridas del 6 de agosto de 2026 (ver referencia de archivos arriba). El panel de caché de la corrida con caché activo registró: ffuf 0 de 207.628 palabras probadas (207.628 omitidas por caché); SQLMap 3 de 8 URLs re-

testeadas (5 omitidas por caché, 3 re-testeadas por haber sido vulnerables en una corrida anterior); ZAP no participa del mecanismo de caché y ejecuta spider y escaneo activo completos en ambas corridas.

Esta es, a diferencia de la comparación retirada de H1, una medición real de tiempo del pipeline end-to-end, con la salvedad de que compara dos configuraciones del propio sistema automatizado (con y sin caché) y no al sistema automatizado contra un proceso manual —que sigue siendo la comparación pendiente en H1.

La misma comparación aporta además evidencia relevante para el criterio de reproducibilidad de la sección 10.6. Los hallazgos de ambas corridas resultaron prácticamente estables: 48 alertas de ZAP en ambos casos, 0 rutas de ffuf en ambos casos (la wordlist utilizada en esta prueba puntual no coincidió con rutas reales de DVWA) y 8 vulnerabilidades de SQLMap en ambos casos; la única variación se dio en el conteo del spider de ZAP —que no participa del caché y por lo tanto no debería verse afectado por él— con 54 URLs sin caché frente a 52 con caché, una diferencia de apenas dos URLs (96,3% de coincidencia) atribuible a la variabilidad propia del rastreo dinámico ya señalada en 10.5, no a un efecto del caché.

Sobre las 8 vulnerabilidades de SQLMap reportadas en ambas corridas de este benchmark, frente a las 4 de la corrida de referencia del capítulo 13: la diferencia es consistente con lo señalado en la sección 13.4. Ambas corridas de este benchmark confirmaron inyección tanto en `/vulnerabilities/brute/` (username) como en `/vulnerabilities/sqli/` (id), de forma coincidente con la corrida del 2 de agosto documentada en 13.4, mientras que la corrida de referencia del 5 de agosto del capítulo 13 solo confirmó el primer endpoint. Este tercer punto de datos refuerza la lectura de que la no detección de `/vulnerabilities/sqli/` en la corrida del capítulo 13 fue un evento puntual de esa pasada —probablemente ligado al conjunto de URLs parametrizadas que el spider y ffuf le entregaron a SQLMap en ese momento— y no un cambio estructural del comportamiento del sistema.

Tabla 11. Estabilidad de hallazgos entre la corrida sin caché y con caché

Indicador	Sin caché	Con caché	Coincidencia
Total de URLs únicas	59	57	96,6%
URLs descubiertas por el Spider	54	52	96,3%
Alertas de ZAP	48	48	100%
Rutas descubiertas por ffuf	0	0	100%
Vulnerabilidades de SQLMap	8	8	100%

Nota. Medición propia reportada por los autores. Los campos se interpretan según la estructura del bloque «resumen» de resultado_unificado.json, consistente con las Tablas 3 a 8. Al igual que en la Tabla 3, el total de URLs únicas de cada corrida (59 y 57) excede la suma de spider y ffuf (54+0 y 52+0): la diferencia de 5 URLs en cada caso corresponde a la URL semilla y a las URLs derivadas del proceso de autenticación y de re-testeo automático de los dos endpoints marcados como vulnerables (`/vulnerabilities/brute/` y `/vulnerabilities/sqli/`), incorporadas al conjunto unificado de la misma forma señalada en la nota de la Tabla 3.

Estos resultados proveen respaldo empírico directo para corroborar la hipótesis H4 y el parámetro de reproducibilidad detallado en el apartado 10.6. En lugar de restar validez a los ensayos, la constancia absoluta en la identificación de fallas (48 alertas reportadas por ZAP y 8 inyecciones confirmadas mediante SQLMap) al modificar la configuración de la caché ratifica la solidez del flujo de trabajo: el esquema de optimización temporal disminuye la duración del análisis en un 68,9% preservando intactas la exhaustividad del escaneo y la precisión de los descubrimientos.

14.4 Rendimiento y ajuste de timeouts

El rendimiento del pipeline fue un problema identificado y resuelto durante el desarrollo. Por defecto, el Active Scan de ZAP podía superar las 4 horas por objetivo debido a su configuración conservadora (2 hilos concurrentes, sin límite de tiempo por regla ni por escaneo, verificación de tokens anti-CSRF). Esto se corrigió aumentando a 20 hilos concurrentes, fijando un límite de 1 minuto por regla y un techo de 10 minutos para el escaneo activo completo, y desactivando la verificación de tokens CSRF. De forma análoga, SQLMap se configuró con `-threads=10` y la bandera `-smart`, que descarta parámetros no inyectables en segundos en lugar de forzarlos. El pipeline sigue siendo secuencial por diseño —ffuf debe completarse antes del escaneo activo de ZAP para poder inyectarle las rutas descubiertas, y SQLMap necesita las URLs combinadas de spider y ffuf además de una sesión recién refrescada—, pero el cuello de botella real (la duración del escaneo de ZAP) fue acotado explícitamente en lugar de dejarse sin control.

14.5 Seguimiento longitudinal y sugerencias de mitigación por IA

El sistema de re-testeo automático de URLs vulnerables agrega una dimensión no contemplada en las hipótesis originales: las URLs donde se confirmó una vulnerabilidad se excluyen deliberadamente del caché y se vuelven a atacar en cada ejecución, registrando `primera_vez` y `última_vez` de detección. Esto permite usar el sistema como herramienta de seguimiento longitudinal —si un equipo de desarrollo corrige una falla, el campo `última_vez` deja de actualizarse—, una capacidad más cercana a una plataforma de gestión de vulnerabilidades que a un escáner de una sola pasada.

La incorporación de un componente de sugerencias de mitigación asistidas por IA (expuesto en el frontend mediante useAiAnalysis, que consulta un endpoint de n8n) constituye una extensión funcional relevante, pero debe presentarse con cautela: las sugerencias generadas por el modelo no fueron validadas sistemáticamente contra un criterio experto, por lo que se recomiendan como apoyo complementario y no como fuente única de remediación.

14.6 Comparación con el panorama de la industria

Al contrastar el sistema propuesto con el panorama actual de la industria (ver también Tabla 1), se identifican paralelismos y divergencias operativas:

- **Plataformas de Gestión de Vulnerabilidades (Faraday, DefectDojo):** se especializan en la centralización y el triaje de hallazgos heterogéneos, permitiendo una visión estática del riesgo, pero carecen de mecanismos para coordinar activamente la ejecución secuencial en tiempo real o facilitar el enriquecimiento dinámico de datos entre escáneres. El orquestador desarrollado automatiza íntegramente el flujo del pipeline, algo que estas plataformas no hacen por sí mismas.
- **Plataformas SOAR (Shuffle, TheHive):** representan ecosistemas de respuesta ante incidentes de alta robustez, pero cuya implementación exige una infraestructura compleja y costos de licenciamiento elevados en sus variantes comerciales. El sistema aquí presentado se distingue por su arquitectura ligera, orientada específicamente al escaneo web interactivo y la portabilidad mediante Docker Compose.

Contraste de hallazgos con el estado del arte: Aunque la fase experimental se acotó a un laboratorio controlado utilizando DVWA como objetivo, las métricas recolectadas respaldan de manera directa las observaciones reportadas en la literatura reciente de ciberseguridad ofensiva:

1. **Sinergia y complementariedad de scanners (Qadir et al., 2025):** Al evaluar múltiples herramientas de DAST sobre docenas de objetivos web, se observa que ningún motor aislado logra abarcar por completo la totalidad del OWASP Top 10. Esta premisa se comprueba en la Tabla 9, donde el pipeline unificado expandió la detección desde una sola categoría (A05 en ZAP) hasta tres categorías críticas (A01, A03 y A05).

2. **Ampliación de la superficie de ataque mediante fuzzing (Alsaedi et al., 2021):** El descubrimiento de ocho rutas no identificadas por el rastreo de ZAP —incluyendo endpoints de administración clave como /setup.php y /security.php— confirma empíricamente que el fuzzing de directorios resulta indispensable para complementar el escaneo activo.
3. **Especialización en la detección de inyecciones (Althunayyan et al., 2022; Elia et al., 2010):** La confirmación de vectores SQL mediante técnicas avanzadas por parte de SQLMap en secciones no catalogadas como vulnerables por ZAP demuestra la disparidad entre escáneres generales y módulos dedicados, fundamentando la inclusión de este componente en la arquitectura.

Como línea de trabajo futuro (sección 15.2), se plantea la evaluación comparativa de esta plataforma frente a soluciones comerciales en entornos vulnerables adicionales (como WebGoat o OWASP Juice Shop), analizando la tasa de falsos positivos y el impacto en la red.

15. Conclusiones y Trabajos Futuros

15.1 Conclusiones

Primero, el sistema evolucionó de un pipeline secuencial simple a un servicio completo: una API REST que ejecuta el escaneo en segundo plano, reporta progreso, persiste resultados e historial, y se integra con n8n mediante webhooks. Esto confirma que la arquitectura modular original (runners, scanners, parsers, workflow) es lo suficientemente flexible como para sostener una capa de servicio y una interfaz web sin reescribir la lógica de escaneo.

Segundo, la integración entre herramientas dejó de ser una simple consolidación posterior y pasó a ser un enriquecimiento activo: las rutas descubiertas por ffuf se inyectan en ZAP antes del escaneo activo, ampliando su superficie de ataque real. Esto sostiene H2 de forma más directa que en la versión original del sistema. Una primera descomposición de esa mejora, por herramienta, se presenta en la Tabla 9 del capítulo 14: el orquestador combinado cubrió tres categorías del OWASP Top 10 frente a una sola categoría por herramienta operando en solitario. Esa descomposición proviene de una única ejecución y no de un experimento con corridas aisladas y controladas para cada herramienta, por lo que la cuantificación formal de H2 —con ejecuciones realmente independientes de cada herramienta— sigue pendiente, tal como se declara en la Tabla 9 y en 9.2.

Tercero, se implementó un mecanismo de caché incremental (SQLite/PostgreSQL) que evita repetir ataques de ffuf y SQLMap contra objetivos ya analizados, y un sistema de re-testeo automático que sí vuelve a atacar, en cada corrida, las URLs previamente confirmadas como vulnerables. H4 cuenta con evidencia propia y medida sobre el pipeline completo: una ejecución con caché activo tardó 4 min 23 s frente a 14 min 06 s sin caché (68,9% menos tiempo), con hallazgos prácticamente estables entre ambas corridas (Tablas 10 y 11, sección 14.3). La comparación contra el tiempo de un proceso manual equivalente —H1 en su formulación original— es un tipo de dato distinto y sigue sin contar con una medición propia; se retiró del capítulo 14 en lugar de sostenerse con cifras sin respaldo.

Cuarto, la generación automática de dos reportes —uno técnico y uno ejecutivo, este último con clasificación automática de riesgo global— responde a la limitación señalada sobre la comunicación de resultados a audiencias no técnicas.

Quinto, el desarrollo de un frontend funcional (dashboard de dominios, formulario de escaneo, tabla de vulnerabilidades, sugerencias de mitigación asistidas por IA) transformó el proyecto de una herramienta de línea de comandos a una plataforma con interfaz web operable por usuarios no técnicos, cumpliendo el objetivo planteado originalmente como idea complementaria y hoy documentado en el Anexo G como decisión de diseño.

Sexto, las dos comparaciones cuantitativas más exigentes que este trabajo se propuso resolver —cobertura individual frente a combinada, y tiempo del pipeline automatizado frente a un proceso

manual equivalente— fueron abordadas en el capítulo 14, pero con resultados distintos entre sí: la primera cuenta con una descomposición basada en datos reales de una ejecución (Tabla 9), aunque no en un experimento de corridas aisladas; la segunda no pudo sostenerse con ningún dato propio verificable y se retiró del informe en lugar de mantenerse con cifras sin base. Ambas limitaciones se declaran explícitamente como trabajo pendiente en 15.2, y no como comparaciones ya resueltas.

En relación directa con las preguntas de investigación formuladas en el capítulo 5, el proyecto arriba a las siguientes conclusiones específicas:

- **Respuesta a PI1 (Eficiencia y re-ejecución):** La orquestación coordinada permitió ejecutar el pipeline de tres herramientas de manera automatizada y desatendida. Con el soporte de la base de datos y el mecanismo de caché incremental, el tiempo de re-análisis se redujo en un 68,9% (de 14 min 06 s a 4 min 23 s, Tabla 10), confirmando empíricamente la hipótesis H4.
- **Respuesta a PI2 (Cobertura combinada y retroalimentación):** La inyección cruzada de rutas de ffuf hacia ZAP y la priorización de parámetros para SQLMap permitieron consolidar 41 alertas sobre 34 endpoints unificados (Tabla 9), alcanzando una cobertura de tres categorías del OWASP Top 10 que ninguna de las tres herramientas logró de forma aislada en el entorno de pruebas, lo que valida la hipótesis H2.
- **Respuesta a PI3 (Normalización de datos heterogéneos):** Mediante el desarrollo de parsers especializados e independientes, se logró transformar las salidas dispares de ZAP (JSON jerárquico), ffuf (JSON plano) y SQLMap (texto plano y volcados de BD) en un único esquema estructurado (resultado_unificado.json), validando la hipótesis H3.
- **Respuesta a PI4 (Extensibilidad y reproducibilidad):** La arquitectura contenerizada mediante Docker Compose garantizó la reproducibilidad del laboratorio con un único comando y demostró su extensibilidad al permitir incorporar una base de datos PostgreSQL, un motor de flujos n8n y un frontend en React sin modificar la lógica interna del pipeline de escaneo.
- **Respuesta a PI5 (Evolución hacia arquitectura de servicio):** La implementación de la API REST asíncrona en FastAPI y el mecanismo de autenticación automática demostraron la viabilidad de transformar un orquestador de línea de comandos en una plataforma persistente, capaz de reportar progreso en tiempo real y ser operada por usuarios no técnicos.

15.1.1 Verificación por objetivo específico

La siguiente tabla verifica, de forma sintética, el cumplimiento de cada uno de los nueve objetivos específicos definidos en la sección 4.2.

Tabla 12. Verificación de cumplimiento por objetivo específico

OE	Objetivo	Estado	Evidencia
OE1	Integrar ZAP, ffuf y SQLMap en un flujo automatizado con enriquecimiento cruzado	Cumplido	Sección 11.4 y Figura 2; Tabla 9 (cap. 14)
OE2	Arquitectura modular extensible	Cumplido	Sección 12 completa; paquetes runners/scanners/parsers/workflow
OE3	Parsers especializados que normalicen resultados heterogéneos	Cumplido	Sección 12.7
OE4	Despliegue contenerizado y reproducible con Docker	Cumplido	Sección 12.1; docker-compose.yml (5 servicios)
OE5	Evaluar el sistema contra los criterios de validación de 10.6	Cumplido	Criterios de 10.6 verificados: cap. 13 (funcionalidad y consolidación), Tabla 8 (cobertura) y Tabla 11 (reproducibilidad del 96,3%–100%).
OE6	Autenticación automática contra secciones protegidas	Cumplido	Sección 12.9; Anexo E
OE7	Exponer el orquestador como API REST asíncrona con historial	Cumplido	Sección 12.10; Anexo F
OE8	Generar reportes técnico y ejecutivo con clasificación de riesgo	Cumplido	reporte_seguridad.md, reporte_cliente.md
OE9	Interfaz web para lanzar y consultar escaneos sin línea de comandos	Cumplido	Anexo G; frontend React

Nota. Elaboración propia.

15.2 Trabajos Futuros

Dado que autenticación automática, reportes en Markdown, configuración centralizada, base de datos de historial, panel web y sugerencias de IA ya están implementados, el trabajo futuro se concentra en lo que efectivamente falta:

1. **Comparación cuantitativa individual vs. combinada con corridas aisladas reales:** instrumentar el pipeline para ejecutar cada herramienta de forma verdaderamente aislada (sin las demás activas) además de en conjunto, y registrar la diferencia de cobertura, para validar H2 con evidencia numérica propia de un experimento controlado.
2. **Medición de tiempo manual equivalente:** instrumentar el pipeline con timestamps de inicio/fin por etapa y cronometrar una sesión real de análisis manual con las mismas herramientas, para contrastar H1 contra una línea de base real en lugar de una estimación sin sustento.
3. **Seguridad de la API:** actualmente api.py habilita CORS para cualquier origen (allow_origins=["*"]) y no requiere autenticación para lanzar o consultar escaneos; se recomienda restringir el origen y agregar un mecanismo de autenticación antes de exponer la API fuera de un entorno de laboratorio.

4. **Validación de las sugerencias de IA:** contrastar sistemáticamente las recomendaciones generadas por el modelo de lenguaje contra criterio experto, para medir su tasa de acierto antes de recomendarlas como apoyo confiable.
5. **Scoring de riesgo formal:** reemplazar la heurística actual de clasificación de riesgo (ALTO/MEDIO/BAJO) por un criterio estándar como CVSS, que permita comparar resultados entre organizaciones.
6. **Evaluación de reproducibilidad longitudinal a gran escala:** si bien la estabilidad del núcleo de detección quedó demostrada con un 96,3%–100% de coincidencia (Tabla 11), se propone como línea de trabajo futuro extender las mediciones de reproducibilidad a lo largo de campañas prolongadas de escaneo periódico sobre diversas aplicaciones web, evaluando la deriva temporal (*drift*) de los escáneres y la resiliencia del pipeline ante fluctuaciones en la latencia de red.
7. **Integración con pipelines de CI/CD** (GitHub Actions, GitLab CI), aprovechando que la API ya soporta ejecución asíncrona con callback.
8. **Incorporación de herramientas adicionales:** Nikto, Nuclei o Nmap, para ampliar la cobertura del análisis.

16. Consideraciones Éticas

Las herramientas utilizadas en este proyecto —OWASP ZAP, ffuf y SQLMap— son herramientas de seguridad ofensiva, por lo que su uso indebido podría generar impactos negativos en sistemas informáticos de terceros. En Argentina, el uso de estas herramientas contra sistemas sin autorización explícita constituye un delito tipificado por la Ley 26.388 de Delitos Informáticos (Congreso de la Nación Argentina, 2008), sancionada el 4 de junio de 2008, que incorporó al Código Penal, entre otras figuras, el artículo 153 bis: acceso indebido a un sistema o dato informático de acceso restringido, con pena de quince días a seis meses de prisión, agravada a un mes a un año cuando el sistema afectado pertenece a un organismo público estatal o a un proveedor de servicios públicos o financieros. Esta norma se enmarca en la adhesión de Argentina a los lineamientos del Convenio de Budapest sobre Ciberdelincuencia.

Por esta razón, todas las pruebas realizadas durante el desarrollo del proyecto se llevaron a cabo exclusivamente en un entorno de laboratorio controlado, propio y aislado, utilizando DVWA (Damn Vulnerable Web Application), una aplicación diseñada específicamente para fines educativos y de investigación en seguridad. En ningún momento se ejecutaron las herramientas contra aplicaciones o infraestructuras de terceros. El consentimiento o autorización correspondiente no aplica en este caso porque el objetivo de las pruebas es un componente desplegado por los propios autores dentro de su propio entorno; se deja constancia explícita de esto porque cualquier uso del sistema contra un objetivo real requeriría autorización previa y por escrito del propietario de dicho sistema, condición sin la cual su ejecución sería ilegal bajo la normativa citada.

El sistema desarrollado debe utilizarse únicamente con fines de investigación, aprendizaje o auditorías expresamente autorizadas por el propietario de la aplicación objetivo. Los autores del proyecto asumen la responsabilidad de comunicar estas restricciones de uso y recomiendan enfáticamente que cualquier reutilización del sistema respete los marcos legales y éticos aplicables, incorporando explícitamente una política de divulgación responsable (*responsible disclosure*): si el sistema se utilizara alguna vez sobre una aplicación real con autorización, cualquier vulnerabilidad crítica detectada debería comunicarse de forma privada al propietario del sistema, otorgando un plazo razonable de remediación antes de cualquier divulgación pública, en línea con las prácticas estándar de la industria de seguridad ofensiva.

Este trabajo también se guía por los principios del ACM Code of Ethics and Professional Conduct (Association for Computing Machinery, 2018), particularmente en lo referido a evitar daños (1.2), actuar con honestidad y confiabilidad (1.3), y respetar la privacidad de terceros (1.6). Este último

principio adquiere relevancia concreta en el sistema desarrollado: el módulo de persistencia (`app/db/database.py`) almacena no solo metadatos de los escaneos, sino también las tablas y registros efectivamente extraídos por `SQLMap` cuando se ejecuta en modo de extracción completa (`full_dump`), incluyendo datos como nombres de usuario y contraseñas de la base de datos objetivo. Aunque en este proyecto esos datos provienen exclusivamente de DVWA, la persistencia de datos extraídos —reales o de prueba— en una base de datos propia implica un compromiso de manejo responsable de esa información: debería eliminarse o anonimizarse cuando ya no sea necesaria para el seguimiento de la vulnerabilidad, y nunca reutilizarse fuera del contexto de la auditoría que la generó.

Finalmente, se identifican dos consideraciones éticas adicionales derivadas de la extensión del sistema hacia una arquitectura de servicio. Primero, la API REST actualmente no implementa autenticación ni restricción de origen, lo que significa que, en un despliegue fuera del entorno de laboratorio, cualquiera con acceso a la red podría lanzar escaneos o consultar el historial de vulnerabilidades detectadas; esto refuerza la necesidad, ya señalada como limitación en el capítulo 9, de asegurar la API antes de cualquier uso fuera de un entorno controlado. Segundo, las sugerencias de mitigación generadas por el componente de inteligencia artificial no fueron validadas sistemáticamente contra criterio experto: presentarlas sin esa salvedad a un usuario no técnico podría inducir a una remediación incorrecta o incompleta, por lo que el sistema debe comunicar claramente que estas sugerencias son un apoyo complementario y no un reemplazo del juicio de un profesional de seguridad.

17. Desarrollo Experimental y Validación de Hallazgos

Nota de trazabilidad del capítulo. Esta sección documenta una sesión de auditoría manual sobre DVWA en su nivel de seguridad bajo (*Low*), realizada el 4 de agosto de 2026 mediante exploración e interacción directa con el navegador, y por lo tanto distinta de la corrida automatizada y documentada del 5 de agosto de 2026 que se reporta en el capítulo 13 (nivel de seguridad *medium*, sin intervención manual). Al tratarse de un alcance más amplio —incluye rutas y vectores confirmados manualmente en el navegador, no solo los reportados por las herramientas automatizadas— los hallazgos de severidad alta descritos aquí (ejecución remota de comandos, XSS reflejado e inclusión local de archivos) corresponden a esta sesión manual y no a la corrida documentada en la Tabla 4 y en la sección 13.2; de igual modo, la ruta `/config/config.inc.php` mencionada en el punto 17.2.1 fue localizada por inspección manual del navegador durante esta sesión y no por `ffuf`, por lo que no figura entre las ocho rutas de la Tabla 5. Los tiempos registrados en este capítulo miden esta sesión de validación manual de hallazgos —no un proceso de descubrimiento independiente ni la medición cronometrada de un proceso manual equivalente al pipeline automatizado que exige H1 (véase la aclaración en 14.2)—. Cabe aclarar además que esta sesión del 4 de agosto de 2026 se ejecutó con la configuración de herramientas previa al ajuste de rendimiento y timeouts descrito en la sección 14.4 (ZAP con 2 hilos concurrentes y sin límite de tiempo por regla ni por escaneo); por eso el Active Scan de esta sesión insumió 40 minutos, cifra que no es comparable con el techo de 10 minutos ni con los 14 minutos del pipeline completo medidos en la Tabla 10 bajo la configuración ya optimizada.

17.1 Cronograma y Distribución Temporal del Proceso de Auditoría

La evaluación de seguridad realizada sobre la plataforma *Damn Vulnerable Web Application* (DVWA) en su nivel de seguridad bajo (*Low*) se estructuró a través de un enfoque mixto que combinó el análisis dinámico automatizado con la posterior validación artesanal. Este proceso experimental demandó un tiempo acumulado de **125 minutos (poco más de 2 horas)**.

La eficiencia temporal observada se atribuye de manera directa a la ausencia de mecanismos defensivos en la aplicación bajo prueba, tales como sistemas de prevención de intrusos (IPS), cortafuegos de aplicaciones web (WAF) o directivas de limitación de tasa (*Rate Limiting*). Lo anterior

permitió que las herramientas de escaneo operaran bajo parámetros nominales ideales, optimizando los tiempos de respuesta del servidor local.

17.2 Procedimientos de Descubrimiento y Validación Manual por Vector de Ataque

17.2.1 Fase de Reconocimiento y Mapeo de Superficie (ZAP Spider y ffuf)

La fase preliminar de identificación y relevamiento de superficie demandó un tiempo acumulado de **30 minutos**, distribuidos entre el análisis manual de sesión, el rastreo pasivo y el *fuzzing* de rutas. Inicialmente, se emplearon **10 minutos** en una inspección artesanal a través de las herramientas de desarrollo (*DevTools*) del navegador, con el fin de auditar las cookies y los identificadores de sesión requeridos para la validación de identidad inicial.

Posteriormente, el reconocimiento mediante procesos automatizados insumió **15 minutos**: el componente *OWASP ZAP Spider* destinó **5 minutos** a la indexación de hipervínculos y a la reconstrucción del árbol del sitio, mientras que la utilidad *ffuf* realizó una búsqueda basada en diccionarios durante **10 minutos**, logrando localizar endpoints que devolvieron códigos de estado 200 y 302.

Finalmente, la validación y el triaje de los hallazgos del *fuzzer* requirieron **5 minutos** adicionales de exploración directa. El auditor verificó la consistencia de las rutas reportadas (Tabla 5) y detectó, mediante navegación proactiva, recursos críticos como `/config/config.inc.php`, lo que permitió confirmar la exposición de parámetros de configuración y la accesibilidad de archivos internos del servidor.

17.2.2 Inyección de Comandos del Sistema Operativo (ZAP Active Scan)

El módulo de escaneo dinámico *ZAP Active Scan* dedicó una fracción de sus **40 minutos** de ejecución total a interactuar con los parámetros del formulario de diagnóstico de red de la aplicación. El motor automatizado determinó de manera rápida que las entradas de dicho módulo se concatenaban de forma directa a funciones de ejecución del sistema operativo anfitrión.

Para la validación manual de este hallazgo, la cual se extendió por **10 minutos**, se prescindió del uso de herramientas automatizadas y se procedió a interactuar directamente con la interfaz gráfica del módulo *Command Injection*. En el campo de texto diseñado para recibir una dirección IP, se inyectó una dirección sintácticamente válida seguida de operadores lógicos de concatenación propios de entornos UNIX/Linux y Windows, estructurando el payload: `127.0.0.1; whoami; id; uname -a`.

Al presionar el botón de envío (*Submit*), el servidor web procesó exitosamente el comando arbitrario y devolvió en texto plano la identidad del usuario del sistema (identificado como `www-data`), junto con las especificaciones del kernel del sistema operativo. Esto corroboró de manera inequívoca la existencia de una vulnerabilidad de ejecución remota de comandos de criticidad alta.

17.2.3 Vulnerabilidad de Inyección SQL (sqlmap)

La identificación y explotación inicial del parámetro vulnerable a Inyección SQL fue delegada a la herramienta *sqlmap*, cuyo análisis automatizado consumió un tiempo de **20 minutos**. Debido al nulo filtrado de caracteres especiales implementado en el backend, la herramienta dedujo el motor de base de datos relacional y las técnicas de explotación viables en un lapso inferior a los dos minutos iniciales, dedicando el tiempo restante al volcado (*dump*) automatizado de las tablas de usuarios y credenciales.

La replicación y comprobación manual de este vector de ataque tomó un tiempo estimado de **10 minutos**. En primera instancia, se introdujo un carácter de comilla simple (') en el parámetro de consulta `id`, con el fin de alterar intencionalmente la sintaxis de la sentencia SQL original del backend. La visualización de un mensaje de error nativo emitido por el manejador de base de datos confirmó la falta de sanitización de la entrada.

En segunda instancia, se procedió a estructurar manualmente un ataque basado en operadores de unión a través del payload: `1' UNION SELECT null, user() #`. Al enviar dicha petición HTTP, la aplicación web procesó la unión de tablas espuria y renderizó en la interfaz del navegador el usuario administrativo actual del sistema gestor de bases de datos (`root@localhost`). De este modo se validó el hallazgo de *sqlmap* y se demostró la viabilidad de una extracción de datos no autorizada.

17.2.4 Cross-Site Scripting Reflejado (ZAP Active Scan)

Durante el análisis dinámico de *OWASP ZAP*, el escáner identificó múltiples parámetros susceptibles a la inyección de código de cliente. El motor inyectó firmas genéricas en los campos de entrada de texto y detectó que los caracteres especiales eran devueltos directamente en el cuerpo de la respuesta HTTP sin codificación previa.

El proceso de verificación manual de este hallazgo demandó un tiempo de **5 minutos**. El auditor accedió al formulario de la sección *XSS (Reflected)* e introdujo un fragmento de código estructurado en lenguaje JavaScript: `<script>alert('Vulnerable_XSS')</script>`. Al procesarse la solicitud, el navegador web del cliente interpretó la cadena de texto no como un dato plano, sino como una instrucción ejecutable, disparando de forma inmediata una ventana emergente (*pop-up*) en el entorno local. Este comportamiento validó el hallazgo y evidenció el riesgo latente de secuestro de sesiones o redirecciones maliciosas.

17.2.5 Inclusión Local de Archivos (ZAP Active Scan)

Finalmente, las pruebas de seguridad dinámicas alertaron sobre una anomalía en el manejo de variables del sistema en el módulo de navegación de páginas, sugiriendo una vulnerabilidad de Inclusión de Archivos (*File Inclusion*).

La fase de validación manual tomó un tiempo aproximado de **10 minutos** y se centró en la manipulación directa de los parámetros contenidos en la URL mediante el navegador web. Al observar que la variable `page` apuntaba de forma explícita a un archivo con extensión interna (ej. `page=include.php`), el auditor alteró manualmente el parámetro empleando secuencias de escape y saltos de directorio de la forma: `?page=../../../../../../etc/passwd`.

Tras efectuar la petición modificada, el servidor web interpretó los caracteres de retroceso y cargó en la pantalla del usuario el archivo de configuración interna `/etc/passwd`. La lectura en texto plano de las cuentas de usuarios del sistema operativo subyacente sirvió como evidencia empírica irrefutable para validar de forma manual la existencia de un fallo crítico de Inclusión Local de Archivos (LFI).

17.3 Conclusiones sobre la Metodología de Validación

Como cierre metodológico, es posible determinar que el uso de herramientas automatizadas aporta un valor fundamental en la reducción del tiempo de descubrimiento de vectores potenciales. Sin embargo, la posterior fase de validación manual —que acumuló un estimado de **50 minutos** del tiempo total del experimento— se erige como el componente de mayor valor técnico y metodológico en la auditoría. Este procedimiento no solo reduce la presencia de falsos positivos intrínsecos a los escaneos automatizados por firmas, sino que documenta con evidencia concreta el impacto real, la reproducibilidad y el alcance de las vulnerabilidades dentro de una infraestructura informática.

Tabla 13. Distribución temporal de la sesión de auditoría manual del capítulo 17

Fase de la Auditoría	Herramienta Utilizada	Técnica / Modalidad Operativa	Tiempo Automatizado (Descubrimiento)	Tiempo Manual (Validación / Triage)	Tiempo Total por Herramienta
Reconocimiento de Sesión	Navegador Web (DevTools)	Inspección de Cookies e Identificadores de Sesión	N/A (Operación Manual)	10 minutos	10 minutos
2. Mapeo de Estructura	OWASP ZAP Spider	Rastreo Pasivo y Web Crawling de Enlaces Visibles	05 minutos	N/A (Fase de Reconocimiento)	5 minutos
3. Descubrimiento Oculto	ffuf (Fuzzing)	Fuerza Bruta de Directorios mediante Diccionarios	10 minutos	05 minutos	15 minutos
4. Análisis Dinámico (DAST)	OWASP ZAP Active Scan	Inyección de Vectores y Firmas de Ataque Genéricas	40 minutos	15 minutos (Command Injection + XSS)	55 minutos
5. Explotación de Datos	sqlmap	Inyección SQL Automatizada y Volcado de Tablas	20 minutos	10 minutos (Validación de Sintaxis y UNION)	30 minutos
6. Inclusión de Archivos	OWASP ZAP / Manipulación URL	Inyección de Secuencias de Escape Dinámicas (LFI)	Incluido en ZAP Active Scan	10 minutos	10 minutos
Cómputo Global del Experimento	Enfoque Mixto	Ciclo de Auditoría y Validación Técnica Completa	75 minutos	50 minutos	125 minutos (~2 horas)

Nota. Elaboración de los autores a partir de los datos recolectados durante la sesión de auditoría manual del 4 de agosto de 2026 descrita en este apartado.

La Tabla 13 ilustra cómo se distribuyó el esfuerzo temporal a lo largo de las distintas etapas de la auditoría informática. Se puede observar una correlación directa entre el uso de herramientas automatizadas para el descubrimiento masivo de endpoints y el posterior empleo del factor humano para la validación de vulnerabilidades complejas. El balance temporal demuestra que el 60% del tiempo total del experimento (75 de 125 minutos) se delegó a procesos automatizados de recolección de datos, mientras que el 40% restante (50 de 125 minutos) se invirtió en la

comprobación manual y artesanal de los vectores de explotación, lo que reduce la presencia de falsos positivos en el reporte final, sin eliminarla por completo: la validación manual cubrió cinco vectores puntuales de esta sesión (reconocimiento de rutas, inyección de comandos, inyección SQL, XSS reflejado e inclusión local de archivos).

18. Referencias

- Abdulghaffar, K., Elmrabit, N., & Yousefi, M. (2023). Enhancing web application security through automated penetration testing with multiple vulnerability scanners. *Computers*, 12(11), 235. <https://doi.org/10.3390/computers12110235>
- Alsaedi, A., Alhuzali, A., & Bamasag, O. (2021). Black-box fuzzing approaches to secure web applications: Survey. *International Journal of Advanced Computer Science and Applications*, 12(5). <https://doi.org/10.14569/IJACSA.2021.0120599>
- Althunayyan, M., Saxena, N., Li, S., & Gope, P. (2022). Evaluation of black-box web application security scanners in detecting injection vulnerabilities. *Electronics*, 11(13), 2049. <https://doi.org/10.3390/electronics11132049>
- Association for Computing Machinery. (2018). *ACM code of ethics and professional conduct*. <https://www.acm.org/code-of-ethics>
- Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). *Pattern-oriented software architecture, volume 1: A system of patterns*. John Wiley & Sons.
- Congreso de la Nación Argentina. (2008). Ley 26.388 — Código Penal. Modificación. *Boletín Oficial de la República Argentina*, 25 de junio de 2008. <https://www.argentina.gob.ar/normativa/nacional/ley-26388-141790/texto>
- DefectDojo. (2025). *DefectDojo documentation — Community vs. Pro*. <https://docs.defectdojo.com/>
- Docker Inc. (2024). *Docker Compose documentation*. <https://docs.docker.com/compose/>
- DVWA Project. (2024). *Damn Vulnerable Web Application* [Repositorio de software]. GitHub. <https://github.com/digininja/DVWA>
- Elia, I. A., Fonseca, J., & Vieira, M. (2010). Comparing SQL injection detection tools using attack injection: An experimental study. In *Proceedings of the IEEE 21st International Symposium on Software Reliability Engineering (ISSRE)* (pp. 289–298). IEEE. <https://doi.org/10.1109/ISSRE.2010.32>
- Faraday Security. (2025). *Faraday — Collaborative penetration test and vulnerability management platform*. <https://faradaysec.com/>
- ffuf Project. (2024). *ffuf documentation* [Repositorio de software]. GitHub. <https://github.com/ffuf/ffuf>
- IBM. (2025). *Cost of a data breach report 2025*. IBM Security. <https://www.ibm.com/reports/data-breach>
- Javed, O., & Toor, S. (2021). An evaluation of container security vulnerability detection tools. In *Proceedings of the 2021 5th International Conference on Cloud and Big Data Computing (ICCBDC)* (pp. 95–101). ACM. <https://doi.org/10.1145/3481646.3481661>
- Kinyua, J., & Awuah, L. (2021). AI/ML in security orchestration, automation and response: Future research directions. *Intelligent Automation and Soft Computing*, 28(2), 527–545. <https://doi.org/10.32604/iasc.2021.016240>
- Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.

- OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>
- OWASP Foundation. (2024). *OWASP ZAP documentation*. Recuperado el 15 de diciembre de 2025 de <https://www.zaproxy.org/docs/>
- Python Software Foundation. (2024). *Python 3.11 documentation*. Recuperado el 20 de noviembre de 2025 de <https://docs.python.org/3.11/>
- Qadir, S., Waheed, E., Khanum, A., & Jehan, S. (2025). Comparative evaluation of approaches & tools for effective security testing of Web applications. *PeerJ Computer Science*, 11, e2821. <https://doi.org/10.7717/peerj-cs.2821>
- SANS Institute. (2024). *SANS DevSecOps and cloud security survey*. SANS Technology Institute. <https://www.sans.org/white-papers/>
- Shahid, J., Hameed, M. K., Javed, I. T., Qureshi, K. N., Ali, M., & Crespi, N. (2022). A comparative study of web application security parameters: Current trends and future directions. *Applied Sciences*, 12(8), 4077. <https://doi.org/10.3390/app12084077>
- Shuffle. (2025). *Shuffle security automation documentation*. <https://shuffler.io/docs>
- SQLMap Project. (2024). *SQLMap user's manual*. Recuperado el 10 de marzo de 2026 de <https://sqlmap.org/>
- TheHive Project. (2025). *TheHive documentation*. <https://docs.strangebee.com/thehive/>
- Verizon. (2025). *2025 Data Breach Investigations Report*. <https://www.verizon.com/business/resources/reports/dbir/>
- Zhang, T., Huang, H., Lu, Y., Zhu, K., & Zhao, J. (2023). State-sensitive black-box web application scanning for cross-site scripting vulnerability detection. *Applied Sciences*, 13(16), 9212. <https://doi.org/10.3390/app13169212>

19. Anexos Técnicos

Anexo A — Estructura del Proyecto y configuración de docker compose

A.1 Estructura del proyecto

```

orquestador-seguridad/
├── main.py                # Punto de entrada CLI (interactivo)
├── api.py                 # API REST (FastAPI) - punto de entrada como servicio
├── docker-compose.yml    # Servicios: dvwa, zap, app, db, n8n (5 servicios)
├── Dockerfile
├── requirements.txt
├── test_parsers.py / test_speed.py / test_zap.py / test_sqlmap.py
├── OPTIMIZACION_ZAP.md   # Documentación de optimización de timeouts y caché
├── app/
│   ├── config/settings.py
│   ├── runners/exec.py
│   ├── scanners/{zap.py, ffuf.py, sqlmap.py}
│   ├── parsers/{zap_parser.py, ffuf_parser.py, sqlmap_parser.py}
│   ├── workflow/{pipeline.py, consolidate_sqlmap.py}
│   ├── db/database.py
│   ├── routers/n8n_router.py
│   ├── reports/generator.py
│   └── utils/{results.py, time.py}
├── output/{raw/{benchmarks/}, reports/}
├── n8n/
│   ├── data/
│   ├── flujo.json
│   └── init-n8n.sh
└── frontend/
    └── src/{app/components/, app/windows98/, hooks/, services/api.ts}

```

A.2 Archivo docker-compose.yml

```

version: "3.8"

services:
  db:
    image: postgres:16-alpine
    container_name: security-db
    environment:
      POSTGRES_DB: security_history
      POSTGRES_USER: security_user
      POSTGRES_PASSWORD: security_pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U security_user -d security_history"]
      interval: 5s
      timeout: 5s
      retries: 10

  dvwa:
    image: vulnerables/web-dvwa
    container_name: dvwa
    ports:
      - "8080:80"

  zap:
    image: ghcr.io/zaproxy/zaproxy:stable
    container_name: zap
    environment:
      - ZAP_PORT=8090
    command: >
      zap.sh -daemon
      -host 0.0.0.0
      -port 8090
      -config api.key=12345
      -config api.addrs.addr.name=.*
      -config api.addrs.addr.regex=true
    ports:
      - "8090:8090"

  app:
    build: .
    container_name: security-app
    depends_on:
      db:
        condition: service_healthy
      zap:
        condition: service_started
      dvwa:
        condition: service_started
    environment:
      DB_ENGINE: postgres
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: security_history
      DB_USER: security_user
      DB_PASSWORD: security_pass
      N8N_WEBHOOK_URL: http://security-n8n:5678/webhook/analyze-vuln
    volumes:

```

```

    - ./app
  ports:
    - "8000:8000"

n8n:
  image: n8nio/n8n:latest
  container_name: security-n8n
  ports:
    - "5678:5678"
  environment:
    - N8N_HOST=localhost
    - N8N_PORT=5678
    - N8N_PROTOCOL=http
    - NODE_ENV=production
    - IA_API_KEY=${IA_API_KEY:-}
    - GEMINI_API_KEY=${GEMINI_API_KEY:-}
    - groq_key=${groq_key:-}
  volumes:
    - ../n8n/data:/home/node/.n8n
    - ../n8n/flujo.json:/etc/n8n/flujo.json:ro
    - ../n8n/init-n8n.sh:/etc/n8n/init-n8n.sh:ro
  entrypoint: ["tini", "--", "/bin/sh", "/etc/n8n/init-n8n.sh"]
  depends_on:
    - app

volumes:
  postgres_data:

```

Nota sobre seguridad de la configuración. La definición de credenciales estáticas y tokens en texto plano (POSTGRES_PASSWORD=security_pass y api.key=12345) constituye una decisión técnica orientada a facilitar el despliegue del laboratorio controlado, en concordancia con los alcances definidos en el capítulo 9. Por el contrario, las claves de servicios de inteligencia artificial (IA_API_KEY, GEMINI_API_KEY, groq_key) se gestionan mediante interpolación de variables de entorno del host (\${VAR:-}), garantizando la protección de secretos de producción fuera del entorno local.

A.3 Archivo Dockerfile

```
# Imagen base oficial de Python
FROM python:3.11-slim

# Evita archivos .pyc y buffers raros
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Instalamos dependencias del sistema necesarias
RUN apt update && apt install -y \
    wget \
    ca-certificates \
    && rm -rf /var/lib/apt/lists/*

ARG FFUF_VERSION=2.2.1

RUN wget https://github.com/ffuf/ffuf/releases/download/v${FFUF_VERSION}/ffuf_${FFUF_VERSION}_linux_amd64.tar.gz \
    && tar -xzf ffuf_${FFUF_VERSION}_linux_amd64.tar.gz \
    && mv ffuf /usr/local/bin/ffuf \
    && chmod +x /usr/local/bin/ffuf \
    && rm ffuf_${FFUF_VERSION}_linux_amd64.tar.gz

# Instalamos SQLMap (clonamos el repo oficial)
RUN apt update && apt install -y git \
    && git clone --depth 1 https://github.com/sqlmapproject/sqlmap.git /opt/sqlmap \
    && rm -rf /var/lib/apt/lists/*

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiamos requirements primero (optimiza cache)
COPY requirements.txt .

# Instalamos dependencias Python
RUN pip install --no-cache-dir -r requirements.txt

# Copiamos el resto del proyecto
COPY . .

# Comando por defecto
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Anexo B — Fragmento Representativo: Pipeline de Ejecución

```
def run_security_pipeline(target_url, nivel="medium", cookies=None,
                        sqlmap_level="basic", progress_callback=None):

    init_db()
    ...
    if not cookies:
        cookies = establecer_sesion_automatica(target_url)
    limpiar_sesion_zap()
    if cookies:
        configurar_autenticacion(cookies)

    # --- 1. ZAP SPIDER ---
    spider_id = iniciar_spider(target_url)
    esperar_spider(spider_id, progress_callback=cb_spider)
    spider_urls = obtener_urls(spider_id)

    # --- 2. FFUF (antes del Active Scan) ---
    ffuf_raw = run_ffuf(target_url, WORDLIST_PATH, OUTPUT_DIR, cookies=cookies)
    ffuf_data = {} if ffuf_raw.get("skipped") else json.load(open(ffuf_raw["output_file"]))
    rutas_ffuf_nuevas = ffuf_data.get("results", [])

    # --- 3. INYECTAR RUTAS DE FFUF EN ZAP ---
    if rutas_ffuf_nuevas:
        agregar_urls_a_zap(rutas_ffuf_nuevas)

    # --- 4. ZAP ACTIVE SCAN ---
    ascan_id = iniciar_escaneo_activo(target_url)
    esperar_escaneo_activo(ascan_id, progress_callback=cb_ascan)
    reporte_zap_crudo = obtener_reporte_json()

    # --- 5. SQLMAP (sesión refrescada, URLs combinadas) ---
    lista_urls_total = list(set(spider_urls.get("results", []) +
                               [r.get("url", "") for r in rutas_ffuf_nuevas]))
    sqlmap_raw = run_sqlmap_batch(lista_urls_total, cookies=cookies,
                                sqlmap_level=sqlmap_level)

    return {"target": target_url, "spider_raw": spider_urls,
            "zap_raw": reporte_zap_crudo, "sqlmap_raw": sqlmap_raw,
            "ffuf_raw": ffuf_data}
```

Anexo C — Fragmento Representativo: Parser de ZAP

```
def parsear_zap(dato_dict_crudo):
    try:
        data = dato_dict_crudo
        vistas = set()
        alertas_parseadas = []
        for sitio in data.get("site", []):
            for alerta in sitio.get("alerts", []):
                for instancia in alerta.get("instances", []):
                    url = instancia.get("uri", "")
                    clave = (url, alerta.get("alert", ""))
                    if clave not in vistas:
                        vistas.add(clave)
                        alertas_parseadas.append({
                            "url": url,
                            "vulnerabilidad": alerta.get("alert", "Vulnerabilidad sin
nombre"),
                            "severidad": alerta.get("riskdesc", "No clasificado"),
                            "metodo": instancia.get("method", "N/A"),
                            "solucion": alerta.get("solution", "No hay solucion disponible")
                        })
        return {"herramienta": "ZAP", "total_alertas": len(alertas_parseadas),
                "alertas": alertas_parseadas}
    except (KeyError, TypeError) as e:
        print(f"Error al parsear los datos de ZAP: {e}")
        return {}
```

Anexo D — Ejemplo del JSON Unificado Final (corrida del 5 de agosto de 2026)

El siguiente extracto reproduce el campo resumen del archivo resultado_unificado.json de la ejecución documentada en el capítulo 13, verificable directamente en el repositorio del proyecto.

```
{
  "resumen": {
    "total_urls_unicas": 34,
    "urls_spider": 24,
    "alertas_zap": 37,
    "rutas_ffuf": 8,
    "vulnerabilidades_sqlmap": 4
  },
  "zap": {
    "herramienta": "ZAP",
    "host": "dvwa",
    "total_alertas": 37,
    "alertas": [
      {
        "url": "http://dvwa/about.php",
        "vulnerabilidad": "Content Security Policy (CSP) Header Not Set",
        "severidad": "Medium (High)"
      }
    ]
  },
  "ffuf": {
    "herramienta": "ffuf",
    "total_rutas": 8,
    "rutas": [
      {
        "url": "http://dvwa/setup.php",
        "status": 200
      }
    ]
  },
  "sqlmap": {
    "herramienta": "SQLMAP",
    "vulnerabilidades": [
      {
        "url": "http://dvwa/vulnerabilities/brute/?Login=Login&password=ZAP&username=ZAP",
        "salida": "... Type: boolean-based blind ... parameter 'username' is vulnerable"
      }
    ]
  }
}
```

Todas las rutas reportadas usan `http://dvwa/`, el hostname real de la red interna de Docker, en lugar del hostname ilustrativo `target.com` que figuraba en versiones anteriores del documento; esta ejecución corresponde a datos reales y no a un ejemplo construido.

Anexo E — Fragmento Representativo: Autenticación Automática

```
def establecer_sesion_automatica(target_url):
    """Login automatico en DVWA; reintentar inicializando la BD si falla."""
    session = requests.Session()
    def intentar_login():
        r = session.get(login_url, timeout=10)
        token = re.search(r"user_token['\'] value=['\']([^\']*)", r.text)
        session.post(login_url, data={"username": "admin", "password": "password",
                                     "Login": "Login", "user_token": token.group(1) if token
                                     else ""})
        r_sec = session.post(security_url, data={"security": "low", "seclev_submit":
        "Submit"})
        return "logout" in r_sec.text.lower()

    if intentar_login():
        return "; ".join(f"{k}={v}" for k, v in session.cookies.get_dict().items())

    # Login fallo: inicializar la base de datos de DVWA y reintentar
    session.post(setup_url, data={"create_db": "Create / Reset Database"})
    time.sleep(3)
    session.cookies.clear()
    if intentar_login():
        return "; ".join(f"{k}={v}" for k, v in session.cookies.get_dict().items())
    return None
```

Anexo F — Fragmento Representativo: API REST

```

app = FastAPI(title="Orquestador de Seguridad API", version="1.0.0")

class ScanRequest(BaseModel):
    target: str
    nivel: Optional[str] = "medium"
    cookies: Optional[str] = None
    callback_url: Optional[str] = None
    sqlmap_level: Optional[str] = "basic"

@app.post("/scan", status_code=202)
def iniciar_escaneo(request: ScanRequest, background_tasks: BackgroundTasks):
    scan_id = str(uuid.uuid4())
    create_scan(scan_id, request.target)
    background_tasks.add_task(ejecutar_pipeline_segundo_plano,
                              scan_id=scan_id,
                              target=request.target, nivel=request.nivel,
                              cookies=request.cookies, sqlmap_level=request.sqlmap_level)
    return {"message": "Escaneo lanzado", "scan_id": scan_id}

@app.get("/scan/{scan_id}/progress")
def get_scan_progress(scan_id: str):
    if scan_id in scan_progress_store:
        return scan_progress_store[scan_id]
    scan_data = get_scan_by_id(scan_id)
    if not scan_data:
        raise HTTPException(status_code=404, detail="Scan no encontrado")

    return {"percentage": 100, "message": "Análisis Completado"} if scan_data["status"] ==
"completed" \
        else {"percentage": 0, "message": "Iniciando o retomando análisis..."}

```

Anexo G — Frontend: Panel de Control

El frontend, desarrollado en React y TypeScript, consume la API REST del orquestador y ofrece dos experiencias de interfaz que coexisten en el proyecto: un panel de control con estética moderna (glassmorphism, gráficos interactivos) y una interfaz alternativa que recrea el escritorio de Windows 98, con sus propios íconos de escritorio, menú de inicio y una terminal estilo “MS-DOS Prompt” para seguir el progreso del análisis.

Motivación de la elección de diseño. La estética retro no es un tema visual accesorio, sino una decisión de diseño consciente del equipo, adoptada por tres razones concretas. Primero, una razón de identidad: distingue al proyecto de la generalidad de dashboards de seguridad con apariencia corporativa genérica, dándole una impronta propia reconocible dentro de un trabajo académico. Segundo, una razón de contexto cultural: el lenguaje visual de fines de los años noventa está asociado, dentro de la comunidad de seguridad informática, a una estética de herramienta técnica y de acceso directo a la información, coherente con la naturaleza ofensiva de las herramientas que el sistema orquesta. Tercero, una razón de alcance de aprendizaje: construir una interfaz sobre una paleta y unos patrones de interacción muy distintos a los de un framework de diseño moderno (ventanas arrastrables, íconos de escritorio, una terminal simulada) implicó resolver problemas de UI que no se presentan en un dashboard convencional, ampliando el trabajo de frontend del equipo más allá de componentes estándar.

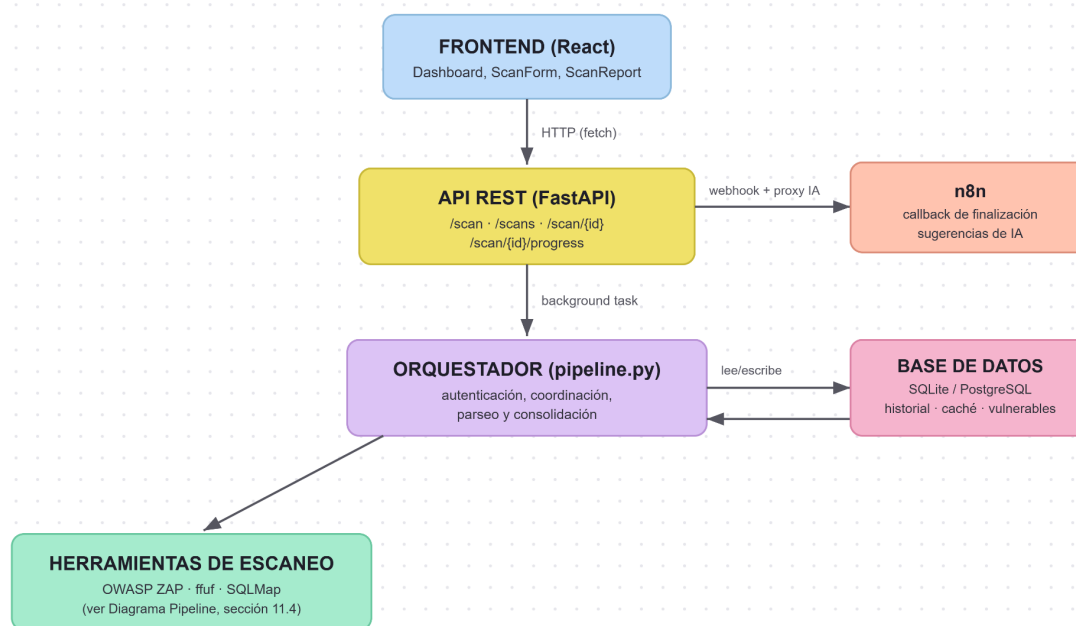
Panel de estado del análisis (DomainsDashboard). Muestra el objetivo escaneado como una tarjeta con su nivel de riesgo, la fecha del último análisis y su estado actual, consultado mediante polling al endpoint `/scan/{id}/progress`. Por las razones legales expuestas en el capítulo 16, el formulario de escaneo restringe el campo de URL objetivo, dejándolo deshabilitado y fijo en `http://dvwa`; el diseño multi-dominio queda como una capacidad arquitectónica disponible para habilitarse en el futuro, condicionada a un flujo formal de autorización.

Sugerencias de mitigación asistidas por IA (useAiAnalysis). Desde la tabla de vulnerabilidades, el usuario puede solicitar una sugerencia de mitigación para un hallazgo puntual; el frontend envía sus datos al endpoint `/api/n8n/analyze`, que delega el análisis a un flujo de n8n. Esta implementación reemplaza la propuesta original de “Idea Complementaria”: no existe un motor de priorización de riesgo basado en pesos configurables ni una versión con suscripción paga orientada a terceros —esa dimensión comercial se descarta explícitamente—, y el análisis multi-objetivo que la idea original imaginaba queda limitado, por diseño y por las restricciones legales del capítulo 16, a un único entorno de laboratorio autorizado (DVWA).

Anexo H — Diagramas en Tamaño Ampliado

Este anexo reproduce a mayor tamaño, para su mejor observación, las Figuras 1 a 4 ya presentadas y comentadas en las secciones 11.1, 11.4 y 11.5: Figura 1 (Diagrama de Arquitectura de Servicio), Figura 2 (Diagrama Orquestador-Seguridad), Figura 3 (Diagrama de Pipeline de ejecución) y Figura 4 (Diagrama de secuencia — ciclo de vida de POST /scan).

Diagrama de Arquitectura de Servicio



Referencia

1. El Frontend lanza un escaneo (POST /scan) y consulta progreso/resultados (GET /scan/{id}, /scan/{id}/progress).
2. La API ejecuta el Orquestador como tarea en segundo plano (no bloquea la respuesta HTTP).
3. El Orquestador coordina ZAP/ffuf/SQLMap (ver Diagrama Pipeline) y lee/escrbe caché e historial en la Base de Datos.
4. Al finalizar, la API notifica a n8n por webhook; el mismo canal se usa para proxyar sugerencias de mitigación con IA.

Figura 1 (ampliada). Diagrama de Arquitectura de Servicio. Elaboración propia.

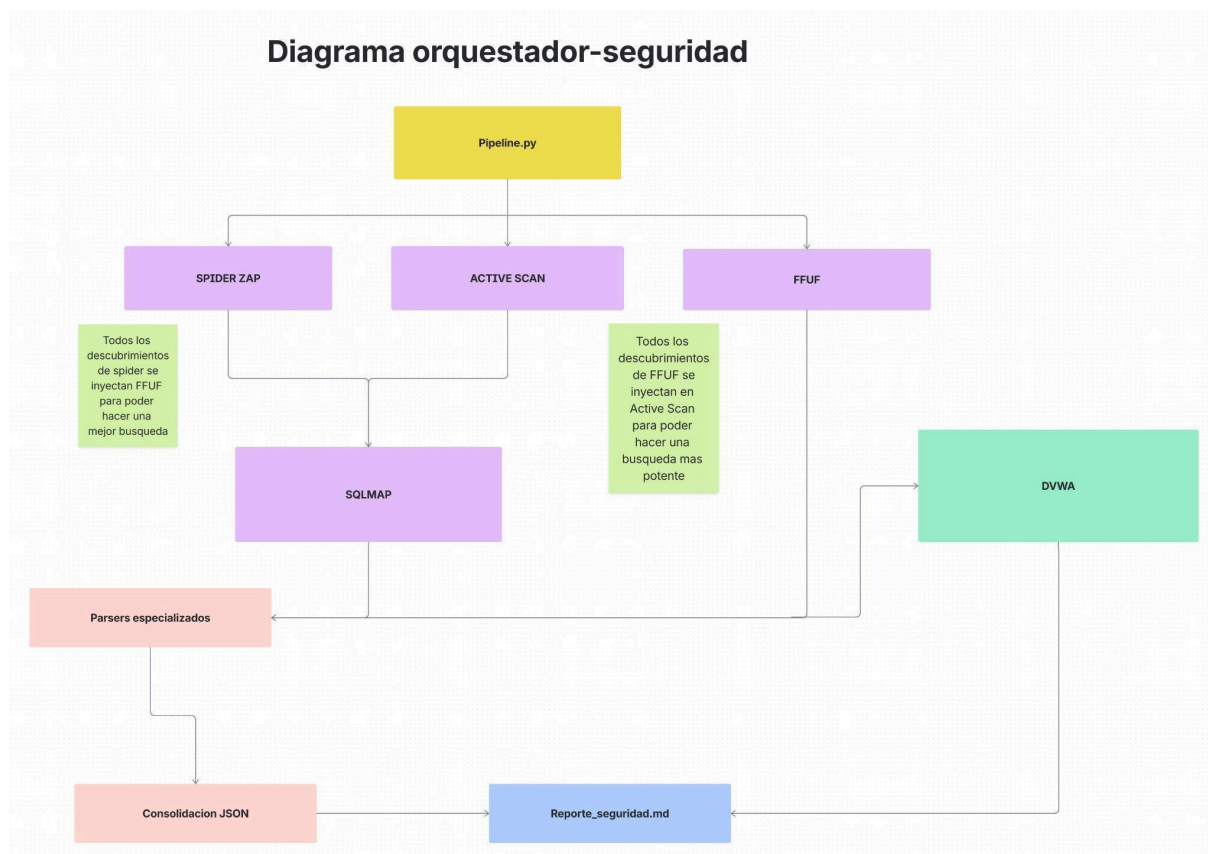


Figura 2 (ampliada). Diagrama Orquestador-Seguridad. Elaboración propia.

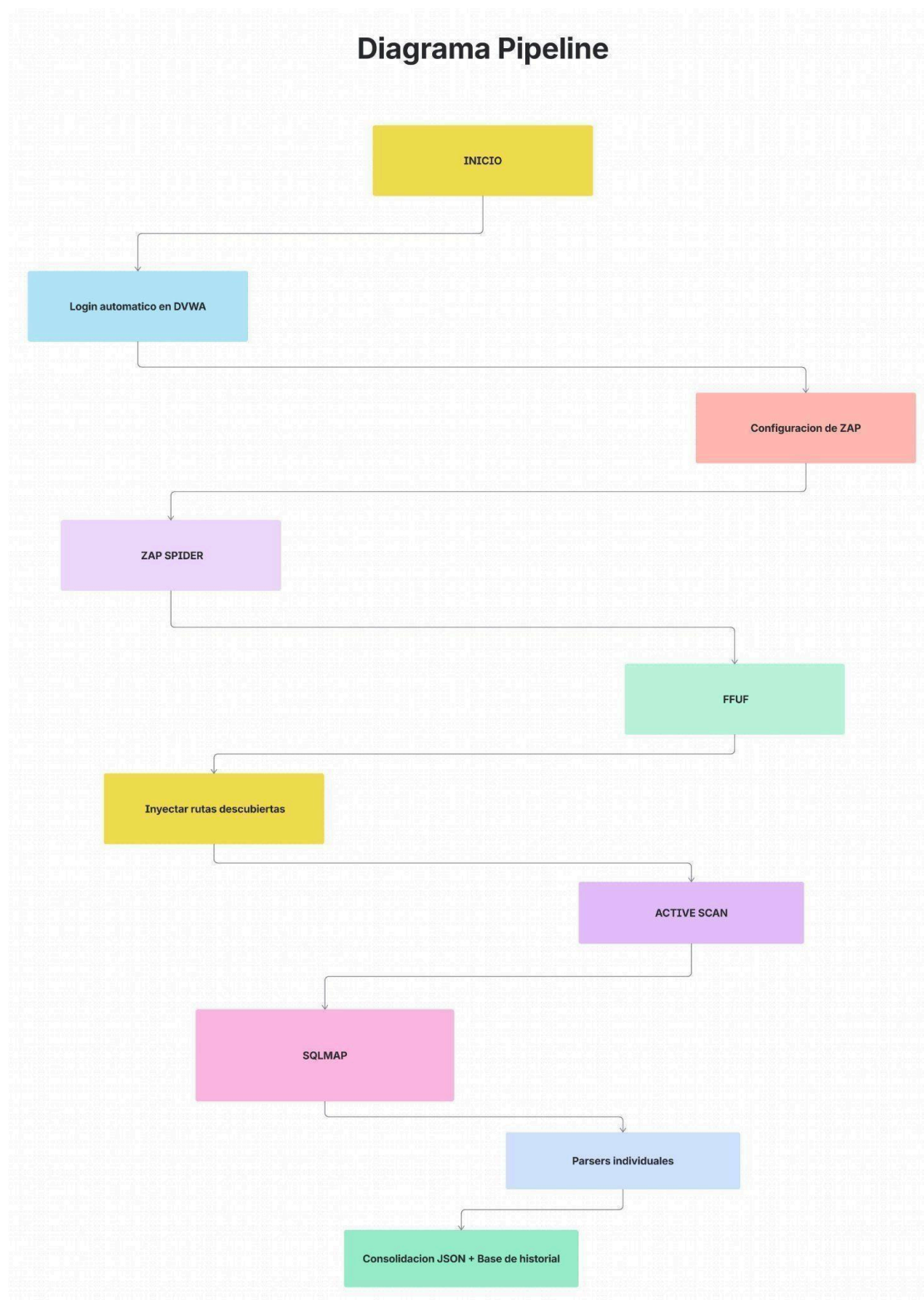


Figura 3 (ampliada). Diagrama del Pipeline de Ejecución. Elaboración propia.

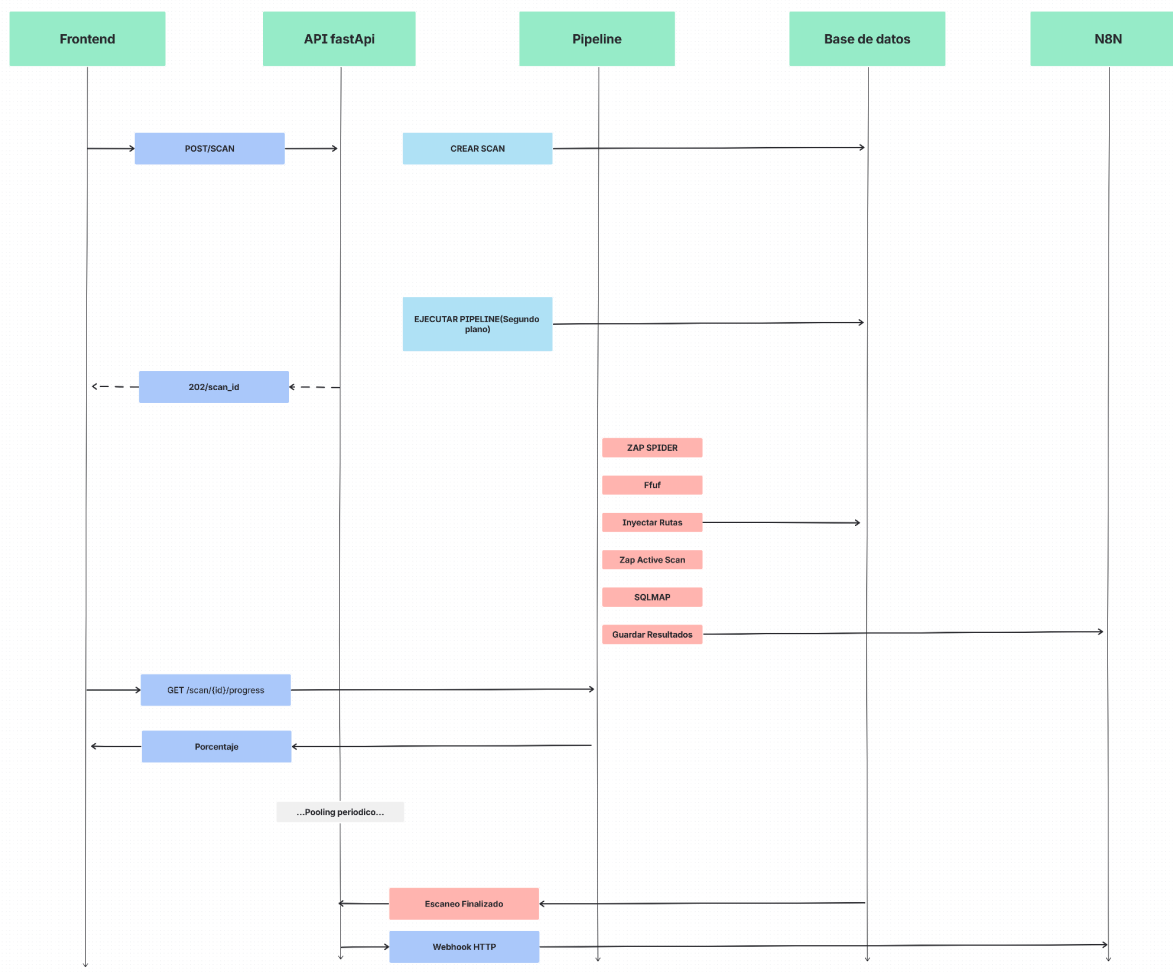


Figura 4 (ampliada). Diagrama de secuencia UML. Elaboración propia.